

Quaderno di lavoro

Come funziona, e quanto costa

Settembre 2026 — dal modello
al datapath reale

Progetto **IPA/eBPF**

inferenza di una rete neurale dentro il kernel, a bordo del pacchetto

Ogni numero di questo quaderno viene da una misura eseguita,
non da una stima. Dove una cosa non è stata verificata, è scritto.

Indice

1	Di che cosa stiamo parlando	4
1.1	L'idea in una pagina	4
1.2	Dove gira: XDP	4
1.3	Le tre pipeline	4
2	La matematica che sta dentro il kernel	6
2.1	Perché i numeri sono interi	6
2.2	I bias vanno riportati nelle unità dell'accumulatore	6
2.3	Il TTL: dividere il prodotto, mai il valore	7
3	Le classi non sono porte	8
3.1	Che cosa significa una classe	8
3.2	Una catena esplicita	8
3.3	Ogni modello ha la sua semantica	8
4	Il motore non è lo scenario	10
4.1	Le dimensioni vengono dalla configurazione	10
4.2	La prova	10
5	Il modello non è quello depositato	12
5.1	La domanda	12
5.2	I modelli sintetici	12
5.3	Il confronto a quattro vie	12
5.4	Il risultato	13
6	Dalla chiamata di sistema al kernel vero	15
6.1	Due banchi	15
6.2	Come il test evita di essere vuoto	15
6.3	Il risultato	16
7	Gli ingressi che dipendono dal nodo	17
7.1	L'interfaccia d'ingresso: una porta logica	17
7.2	L'identità del nodo	17
7.3	La prova che contano	17
8	Il verificatore paga i cammini	19
8.1	Che cosa fa il verificatore	19
8.2	Il primo strato: feature fuori, neuroni dentro	19
8.3	La ReLU: un bivio per neurone	19
8.4	La ReLU senza bivio: il risultato	21
8.5	La ReLU senza bivio: che cosa costa	21

9	I numeri, e che cosa dicono davvero	23
9.1	La tabella	23
9.2	Primo risultato: la dimensione non predice la velocità	23
9.3	Secondo risultato: quanto valgono le latenze assolute	24
9.4	Terzo risultato: la genericità ha un costo di verifica	25
9.5	Quarto risultato: togliere il compilatore dal nodo	25
9.6	Che cosa questi numeri non dicono	26
10	Come cresce il costo	27
10.1	La domanda	27
10.2	Come è impostata la misura	27
10.3	Profondità: chi paga in spazio e chi in tempo	28
10.4	Aggiornare il modello: con l'oggetto precompilato, pari	29
10.5	Larga o profonda? Dipende da chi la esegue	30
10.6	Congelare il nodo: conviene?	32
10.7	Congelare quali porte esistono	35
10.8	Due larghezze, due risposte opposte	36
10.9	I pesi contano, ma solo per una delle tre	37
10.10	Che cosa questi grafici non dicono	38
11	Il costo del programma, misurato da solo	39
11.1	La campagna sulle architetture	39
11.2	Il problema: le MAC nominali non predicono la latenza	39
11.3	La soluzione: contare le MAC che vengono eseguite	40
11.4	Due banchi, lo stesso programma, lo stesso numero	40
11.5	Il programma sul percorso vero: tre marcature	41
12	Il test reale: quanto costa un pacchetto al nodo	42
12.1	La domanda, in parole semplici	42
12.2	Com'è fatto il banco	42
12.3	Come si misura, e perché così	43
12.4	I due generatori	43
12.5	Com'era il test nella macchina virtuale, e com'è adesso	44
12.6	I limiti del test, con esempi	45
12.7	Il risultato: quanto regge il nodo	47
12.8	Il costo del nodo, con pacchetti grezzi	48
12.9	Uno scrittore per coda	49
12.10	Al crescere del bit rate	50
12.11	Che cosa questo numero non dice	54
13	La macchina di laboratorio, sotto controllo	55
13.1	Un processore che non sta fermo	55
13.2	Le condizioni, messe e tolte	55
13.3	La frequenza scritta non è la frequenza vera	56
13.4	Quale frequenza	56
13.5	Il risultato: misure che si ripetono	57
13.6	Prima e dopo: che cosa è cambiato	57
13.7	Come si rifà tutto	58

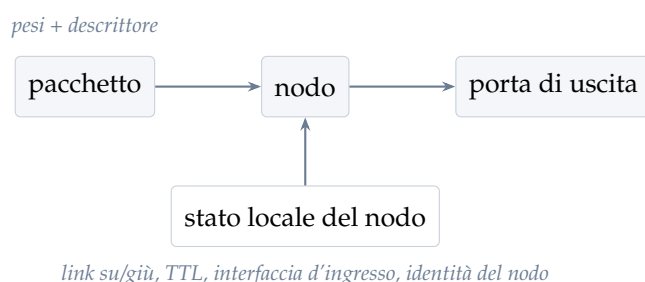
14 Altri modelli, altre reti	59
14.1 La domanda	59
14.2 Scenario e modello	59
14.3 Quando un modello può girare su una rete	59
14.4 Come è costruito	60
14.5 Che cosa controllano i test, adesso	61
14.6 La prova che la strada nuova non cambia niente	61
14.7 I risultati	62
14.8 Che cosa non fa ancora	62
15 I numeri di oggi, in una pagina	63
16 Quel che resta aperto	64
17 I comandi	65
17.1 La macchina	65
17.2 Correttezza: il modello fa quello che deve	65
17.3 Kernel: si carica, e quanto costa	65
17.4 Altri modelli, altre reti	66
17.5 Datapath reale: il pacchetto attraversa davvero	66
17.6 Analisi parametrica: come cresce il costo	66
17.7 Il test reale	66
17.8 Costi specifici e oggetto precompilato	67
17.9 L'ordine consigliato	67

Di che cosa stiamo parlando

1.1 L'idea in una pagina

Un router normale decide dove mandare un pacchetto consultando una tabella riempita da un protocollo di routing. L'idea di **IPA** (*Intelligent PAckets*) è diversa: il pacchetto **porta con sé** una piccola rete neurale, e ogni nodo che attraversa la esegue per decidere l'uscita.

Niente segnalazione, niente tabella condivisa: la decisione nasce dai dati che il nodo ha sotto mano in quel momento.



Il modello è una rete piccola: quattro tipi di ingresso, due strati nascosti da quattro neuroni, sette uscite. In tutto **319 pesi**. Deve stare in un header ed essere eseguibile in decine di nanosecondi.

1.2 Dove gira: XDP

Il codice gira in **eBPF** agganciato a **XDP**, il punto più precoce in cui il kernel Linux permette di toccare un pacchetto: prima che venga costruita la struttura `sk_buff`, quindi prima di quasi tutto il resto dello stack di rete.

Questo impone un vincolo forte, che spiega metà delle scelte di progetto: un programma eBPF deve essere **dimostrato sicuro** da un componente del kernel chiamato *verificatore* prima di poter girare. Niente cicli di lunghezza ignota, niente accessi non controllati alla memoria, niente ricorsione. Tutto ciò che nel documento sembra contorto — cicli srotolati, tetti dichiarati a priori, tabelle al posto di calcoli — nasce da lì.

1.3 Le tre pipeline

Lo stesso modello è implementato in tre modi, per misurare un compromesso.

	i pesi stanno	cambiare modello
P1 hardcoded	scritti nel codice C	ricompilare
P2 template	in una mappa	scrivere la mappa
P3 modular	in una mappa, uno strato per programma	scrivere la mappa

P1 è veloce perché il compilatore vede i pesi e li semplifica; è rigido perché ogni modello nuovo richiede una ricompilazione. P2 e P3 sono l'opposto. Questo quaderno misura il compromesso.

P1 si costruisce **prima** (*ahead of time*, AOT): il C con i pesi scritti dentro si compila con clang su una macchina qualunque, e al nodo arriva l'oggetto pronto, che un piccolo caricatore in C apre e attacca. Il caricatore lascia vuote le tabelle e le «appunta» nel file system del kernel; il piano di controllo in Python le riempie con le interfacce vere del nodo, poi dice al caricatore di attaccare il programma. Ogni cifra di P1 in questo quaderno è di quell'oggetto, attraverso un modulo solo (`ipa/p1_aot.py`), e anche il controllo rapido `execute_pipeline.py -method hardcoded -verify-only` carica quell'oggetto: lo genera, lo fa passare dal verificatore e lo esegue una volta, senza agganciarlo.

La matematica che sta dentro il kernel

2.1 Perché i numeri sono interi

Nel kernel non ci sono numeri in virgola mobile: eBPF lavora con interi. Il modello però è addestrato in virgola mobile. Serve una conversione, e si chiama **quantizzazione**.

Ogni peso reale w diventa un intero a 8 bit:

$$w_{\text{int}} = \text{round}(w \cdot s)$$

dove s è il **fattore di scala**, uno solo per tutta la rete.

Perché il risultato stia in un byte con segno serve

$$s \cdot \max_i |w_i| \leq 127.$$

Sul modello depositato $\max |w| = 5,098$, quindi $s \leq 24,9$. Due scelte convivono nel progetto: $s = 24$ (il massimo intero ammesso) e $s = 16$ (la potenza di due più vicina per difetto, che rende le divisioni degli shift).

Superare quel limite non dà un errore: dà un *overflow silenzioso*. I pesi grandi si troncano e il modello sbaglia senza che nulla lo segnali. Per questo i test distinguono le scale *rappresentabili* da quelle che saturano, e misurano solo sulle prime.

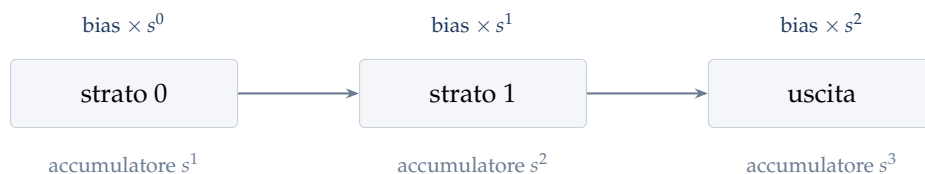
2.2 I bias vanno riportati nelle unità dell'accumulatore

Quando moltiplichiamo un ingresso per un peso quantizzato, il risultato porta un fattore s in più rispetto al valore reale:

$$x \cdot w_{\text{int}} = x \cdot w \cdot s.$$

Se impiliamo gli strati, quel fattore si accumula. Dopo $\ell + 1$ moltiplicazioni l'accumulatore dello strato ℓ porta $s^{\ell+1}$.

Ma il **bias** è memorizzato come un peso qualsiasi, cioè con un solo fattore s . Sommarlo così com'è significherebbe sommare una quantità sistematicamente troppo piccola. Va riportato nelle unità dell'accumulatore:



La regola è semplice: allo strato ℓ (contando da zero) il bias va moltiplicato per s^ℓ . In P1 il moltiplicatore si calcola alla generazione del codice e il bias entra già scalato come letterale; in P2 e P3 il programma lo calcola a runtime dalla scala del modello.

Perché i test spengono i link

Con **tutti i link attivi** gli accumulatori sono grandi, e anche un bias sottopesato di $s^2 = 576$ volte non sposterebbe mai il massimo. Basta spegnere qualche link perché gli accumulatori si riducano e un errore del genere decida l'esito. Per questo le prove di correttezza variano lo stato dei link, non solo il TTL.

2.3 Il TTL: dividere il prodotto, mai il valore

Il modello è addestrato con il TTL *normalizzato*, cioè ttl/T con $T = 30$ il valore iniziale. Con i numeri interi

$$\frac{\text{ttl}}{30} = 0 \quad \text{per ogni } \text{ttl} < 30,$$

e la feature sparirebbe. La divisione va quindi applicata al *prodotto*:

$$\left\lfloor \frac{\text{ttl} \cdot w_{\text{int}}}{T} \right\rfloor \quad \text{e non} \quad \left\lfloor \frac{\text{ttl}}{T} \right\rfloor \cdot w_{\text{int}}.$$

Nel C generato si legge esattamente così:

```
(((_s64)_ttl * 11LL) / 30LL)
```

La scala T è del **modello**, non del codice: un modello addestrato su $\text{ttl}/16$ va eseguito con 16. Viaggia quindi nel descrittore del modello (il campo `scale` di `struct feat_ent`, un byte); P1 la compila come letterale, P2 e P3 la leggono dalla mappa a ogni pacchetto, e con 0 ricadono sul default compilato.

In P2 e P3 il divisore è quindi una variabile, e **BPF non ha la divisione con segno**: il numeratore invece il segno ce l'ha, perché i pesi possono essere negativi. Il datapath divide il valore assoluto senza segno e rimette il segno, con troncamento verso lo zero — lo stesso arrotondamento del riferimento Python, perché sui negativi le due implementazioni coincidono solo se arrotondano allo stesso modo.

Le classi non sono porte

3.1 Che cosa significa una classe

La rete produce sette numeri; il più grande vince. Ma *che cosa significa* la classe vincente?

Il significato si può *dedurre* — «la classe 6 è lo scarto», «DROP è l'ultima classe», «la classe k esce dall'interfaccia k » — e sul modello depositato ognuna di queste deduzioni sembrerebbe funzionare, perché lì le porte sono numerate come le classi. Su un altro modello darebbe risposte plausibili e sbagliate, senza alcun errore.

3.2 Una catena esplicita

Il significato è quindi **dichiarato**, mai dedotto, e passa per una catena di quattro passi, ognuno con un responsabile diverso:



- **classe**: quello che il modello ha calcolato;
- **azione**: FORWARD, DROP o UNUSED — una classe mai addestrata esiste ed è dichiarata tale;
- **porta logica**: un numero che il *modello* conosce;
- **ifindex**: l'interfaccia vera, che solo il *nodo* conosce e che è diversa su ogni macchina.

Nel modello depositato la dichiarazione è:

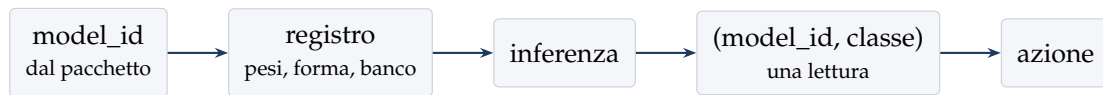
classe	azione	porta logica
0-4	FORWARD	0-4
5	DROP	—
6	UNUSED	—

Classe e porta sono due spazi di indici distinti anche nel datapath: il contatore per classe si scrive con l'indice della classe, la tabella delle uscite si legge con quello della porta. Sul modello depositato coincidono, ma il codice non ci conta.

Con la semantica dichiarata, un modello diverso da quello depositato **smette di funzionare rumorosamente** invece di continuare a girare dando risposte plausibili e sbagliate.

3.3 Ogni modello ha la sua semantica

In P2 e P3 un solo programma per architettura serve più modelli, caricati come dati. Modelli diversi possono dare alla stessa classe significati diversi, e quindi la chiave della tabella classe → azione è la coppia (model_id, classe).



- **Una tabella ad accesso diretto, non una tabella hash:** $256 \text{ modelli} \times 2 \text{ banchi} \times 32 \text{ classi}$. Una sola lettura per pacchetto, e nessun ciclo sui modelli registrati.
- **Due banchi per modello.** Ricaricare un modello scrive il banco inattivo, poi un'unica scrittura del registro cambia insieme il numero di classi e il banco in uso. Un pacchetto vede la semantica vecchia o quella nuova, mai una tabella scritta a metà.
- **Perché non dentro il registro,** che il programma ha già letto: usare la classe come indice dentro quel valore costringe il verificatore a seguirne il valore esatto attraverso tutto l'argmax, e smette di potare i cammini. Misurato: `layer_hidden` di P3 non si carica.
- **Senza un valore vivo in più.** Il primo strato di P3 sta al limite dei 512 byte di stack nelle sue versioni strumentate: la chiave viaggia allora nella variabile che dice «questo è l'ultimo strato».

La prova. Modelli costruiti in modo che il TTL scelga la classe, così ogni classe di ogni modello è raggiungibile: un modello solo, due con la stessa semantica, due con semantiche opposte su ogni classe a pacchetti alternati, un modello inesistente o rimosso, numeri di classi diversi, un modello ricaricato con un'altra semantica. **53/53** su P2 e P3. Con una tabella condivisa simulata gli stessi test falliscono dove devono: il test distingue le due cose.

Il datapath non se ne accorge: la chiave per modello è una lettura come quella per sola classe, e la latenza non cresce da 1 a 8 modelli registrati. Il prezzo è in memoria: 128 KiB fissi per pipeline, riservati per 256 modelli anche quando ne è caricato uno solo.

Il motore non è lo scenario

4.1 Le dimensioni vengono dalla configurazione

Il modello depositato è addestrato su **Germany50**, una rete di 50 router usata come riferimento in letteratura, con due host attaccati. Due numeri ne descrivono la forma:

$$n_{\text{nodi}} = 52, \quad n_{\text{interfacce}} = 6.$$

Il primo è $50 + 2$. Il secondo è il **grado massimo** della rete: il nodo con più vicini. Non è il grado di un nodo qualsiasi — tutti gli altri riempiono di zeri gli slot che non usano.

Questi numeri determinano la larghezza dell'ingresso:

$$n_{\text{in}} = \underbrace{6}_{\text{link}} + \underbrace{6}_{\text{iface}} + \underbrace{1}_{\text{ttl}} + \underbrace{52}_{\text{nodo}} = 65.$$

Il motore non li conosce. Il progetto è diviso in tre strati:



La dipendenza punta **solo verso l'alto**. Il motore non sa che cosa sia Germany50.

Le dimensioni si risolvono in ordine: una variabile d'ambiente, poi un file di sistema, poi il blocco `trained_on` del descrittore del modello. Se nessuna delle tre risponde, il motore **si ferma** e dice dove ha cercato.

Un default comodo è il modo più efficace di nascondere un'assunzione. Preferire un errore esplicito a un valore plausibile è una scelta di progetto: il valore plausibile non si manifesta mai come errore, solo come risultato sbagliato.

4.2 La prova

Il motore genera un datapath corretto per cinque topologie che non hanno niente a che vedere con Germany50:

iface	odi	strati	n_{in}	n_{out}	vettore link
3	8	4-4	15	4	v[3]
4	16	6	25	5	v[4]
5	24	4-4-4	35	6	v[5]
2	6	3-3	11	3	v[2]
6	52	4-4	65	7	v[6]

Nessuna riga di C scritta a mano cambia fra una riga e l'altra. `test_fabric.py -sweep:`
34/34.

Il modello non è quello depositato

5.1 La domanda

Un datapath che calcola la cosa giusta su un modello può calcolarla per caso: perché quel modello ha una forma particolare, perché i suoi pesi cadono in un intervallo comodo, o perché una costante scritta nel codice coincide con un suo parametro. Con un modello solo, la differenza fra “corretto” e “corretto qui” non si vede. Servono altri modelli.

5.2 I modelli sintetici

Otto scenari generati, con pesi casuali e nessun rapporto con il checkpoint depositato. Coprono un asse ciascuno:

Scenario	Forma	Pesi	scala del TTL
ones	11-2-3	33	8
small	15-4-4	84	16
mixed	18-6-5	149	30
ipa_like	65-4-4-7	319	30
ipa_ttl16	65-4-4-7	319	16
sparse	65-4-4-7	319	30
deep	65-4-4-4-7	339	30
large	117-8-8-9	1097	64

Il generatore non produce solo numeri a caso: rispetta la **struttura** delle feature, che è la parte che un datapath può sbagliare. Ogni feature dichiara il suo tipo — `real` (un valore in un intervallo, che il datapath porta come intero diviso per una scala), `integer`, `binary` (un vettore di 0/1), `onehot` (esattamente un 1) — con i suoi estremi, la sua scala e i suoi vincoli. Ogni scenario porta con sé **1 000 vettori d’ingresso**, nella versione intera e in quella a virgola mobile, e le predizioni attese per entrambe.

Gli artefatti non stanno in git: sono rigenerabili byte per byte a parità di seme, e il test li rigenera da solo se non li trova.

5.3 Il confronto a quattro vie

Compilare e passare il verificatore dimostra che il programma esiste, non *quale classe sceglie*. E due confronti non bastano, perché un disaccordo ha più di una causa possibile e le cause chiedono rimedi diversi. Le vie sono quattro:

Confronto	Che cosa isola
float vs int8 (Python)	l'errore di quantizzazione : il modello a interi decide diversamente da quello a virgola mobile. Può legittimamente non essere 100%.
int8 (Python) vs int8 (riferimento)	controprova : due implementazioni intere indipendenti del progetto devono concordare fra loro, o non è chiaro quale sia il riferimento.
int8 (riferimento) vs int8 (eBPF)	se il datapath è esatto rispetto alla configurazione che gli è stata data .
int8 (Python) vs int8 (eBPF)	l'errore di implementazione . Deve valere 100%: un solo disaccordo qui è un difetto del datapath.

C'è anche un quinto confronto che non ha bisogno del kernel: `p1_c_eval.py` legge il C generato per P1 *come testo* e lo valuta con le regole degli interi del C, logit per logit, contro il riferimento Python. I numeri vengono dai letterali del sorgente, mai dalla lista dei pesi: un generatore che emettesse un moltiplicatore sbagliato darebbe un'altra risposta. Fatto girare su codice alterato apposta, se ne accorge.

Perché non i 1 000 vettori depositati

Il datapath non accetta un vettore d'ingresso arbitrario: lo *costruisce* da ciò che ha sotto mano — le mappe dense, il TTL del pacchetto, la porta d'ingresso risolta, l'indice del nodo. Un one-hot node diverso a ogni campione vorrebbe dire riconfigurare il nodo a ogni campione.

Si campiona quindi nello spazio che il datapath sa davvero esprimere, e il vettore Python si costruisce allo stesso modo. Confrontare due implementazioni su ingressi che una delle due non potrebbe mai ricevere non direbbe niente su nessuna delle due. Per la stessa ragione il banco comunica al kernel la porta d'ingresso che il riferimento usa: `BPF_PROG_TEST_RUN` senza contesto fa vedere al programma sempre l'interfaccia di prova del kernel.

5.4 Il risultato

Trecento ingressi per scenario, seme fisso.

Scenario	float vs int8	int8 Python vs eBPF (P1)	decisioni
ones	100,00 %	100,00 %	300/300
small	98,00 %	100,00 %	300/300
mixed	100,00 %	100,00 %	300/300
ipa_like	97,67 %	100,00 %	300/300
ipa_ttl16	98,00 %	100,00 %	300/300
sparse	99,33 %	100,00 %	300/300
deep	98,67 %	100,00 %	300/300
large	99,67 %	100,00 %	300/300

Equivalenza numerica esatta. P1 la dà su 8 scenari su 8, P2 e P3 su 7 su 7 (large non si applica: supera la tabella dei pesi di P2 e le uscite di P3). La colonna di sinistra — fra 97,67 % e 100 % — è la quantizzazione, cioè una proprietà del modello e della scala scelta, non un difetto.

La scala del TTL viene letta davvero. Sugli scenari con scala diversa da 30 la scala dichiarata sposta le decisioni (41 casi su 300 per `ipa_ttl16`, 89 per `small`). Controllo negativo in P2 e P3: riscritta nella mappa del nodo la scala al default, il datapath si stacca dal riferimento *esattamente* su quei casi, e con la scala giusta torna identico.

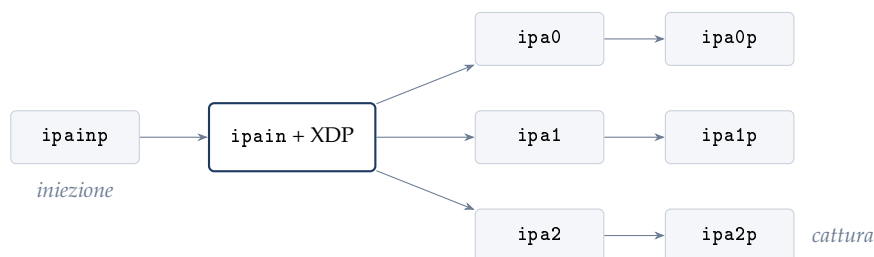
Un'affermazione di genericità — “il descrittore si legge a runtime, quindi qualunque modello va bene” — va verificata **voce per voce**: la forma, il numero di feature, gli offset e la scala sono quattro affermazioni, e ognuna ha bisogno del suo modello di prova.

Dalla chiamata di sistema al kernel vero

6.1 Due banchi

Il primo banco è una chiamata di sistema, `BPF_PROG_TEST_RUN`: si dà al programma un buffer, lo si esegue N volte, e il kernel riporta quanti nanosecondi in media. È utile e onesto per confrontare le pipeline fra loro, ma non tocca un'interfaccia, non ha un driver, non consegna niente. Il programma può dire "redirigerei sulla porta 3" senza che nessun pacchetto ne esca.

Il secondo banco è fatto di coppie di interfacce virtuali (veth) create sull'host: abbastanza vicino al vero da far girare XDP nella modalità in cui gira su un nodo.



Si inietta un frame da un capo, il programma XDP gira davvero, e si legge da quale interfaccia è uscito.

Due dettagli che sembrano minori e non lo sono

Modalità di attacco. XDP si può agganciare in due modi: *native*, dentro il driver, prima che esista la `sk_buff`; oppure *generic*, dopo. Il default è *native*, la modalità *davvero in effetto* viene riletta dal kernel dopo l'aggancio, e un fallimento è un errore, non una retrocessione silenziosa: una misura fatta in *generic* senza saperlo non si potrebbe attribuire a niente.

Il vincolo delle veth. Redirigere *verso* una veth funziona solo se il lato ricevente ha a sua volta un programma XDP agganciato: il kernel alloca le code necessarie solo in quel caso. Senza, il redirect fallisce in silenzio e il pacchetto sparisce — indistinguibile da un modello che decide di scartare tutto. Il banco aggancia quindi un programma minimo a ogni capo.

6.2 Come il test evita di essere vuoto

Un test che non può fallire non è un test. Tre scelte del banco servono a questo:

- **Cerca un ingresso per ogni classe**, invece di spazzare alla cieca. Con tutti i link attivi il modello depositato risponde sempre con la stessa classe per qualunque TTL: variare il solo TTL ripeterebbe un caso solo. È `link_state` la feature che sposta la decisione.
- **Filtra la cattura** sui campi che il datapath non riscrive, e disabilita IPv6 sulle interfacce del banco: una veth appena creata manda traffico suo (IPv6 multicast), che altrimenti potrebbe essere contato come la risposta della pipeline.

- **Ha un ingresso dedicato**, separato da tutte le porte: se l'ingresso fosse anche una porta d'uscita, un frame inoltrato su quella porta non si distinguerebbe da quello appena iniettato.

E lo scarto si verifica dal *contatore*, non dal silenzio: "non è arrivato niente" è anche ciò che si osserva quando il redirect si perde.

6.3 Il risultato

Prova	esito
consegna su rete di veth, tutte le pipeline, 6 classi su 7	29/29
consegna della P1 come si deploia (oggetto AOT, mappe appuntate)	11/11
cinque topologie diverse	34/34

La settima classe è *UNUSED*: dichiarata, mai addestrata, e non ci sono ingressi che la raggiungano.

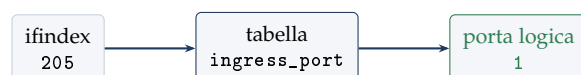
Gli ingressi che dipendono dal nodo

Due dei quattro ingressi del modello non vengono dal pacchetto ma dal nodo: *da quale interfaccia è arrivato il pacchetto*, e *quale nodo* lo sta elaborando. Tutti e due si risolvono con una tabella scritta dal piano di controllo quando il nodo entra in servizio.

7.1 L'interfaccia d'ingresso: una porta logica

L'ingresso è un one-hot di sei posizioni. Il kernel però identifica le interfacce con un `ifindex` assegnato in ordine di creazione: su un sistema appena avviato valgono 2, 3, 4; su una macchina che ha creato qualche interfaccia virtuale 205, 217, 229. Usarlo come posizione nel vettore non accenderebbe mai un bit.

Il datapath lo traduce allora in **porta logica** con una tabella a runtime, riempita dal piano di controllo leggendo le interfacce del nodo:



È il **simmetrico** della tabella usata in uscita: là "quale interfaccia realizza la porta che il modello ha scelto", qui "quale porta è l'interfaccia da cui il pacchetto è arrivato". Entrambe sono fatti del nodo, entrambe si risolvono quando il nodo esiste. Un'interfaccia che non è una porta del modello non ha voce, e il one-hot resta spento.

7.2 L'identità del nodo

L'ingresso più grande del modello è *quale nodo sta elaborando il pacchetto*: su Germany50 un one-hot di 52 posizioni, **208 dei 319 pesi**.

Non si può prendere dal pacchetto. Il campo `model_id` dell'intestazione dice *quale modello* il pacchetto chiede di usare, e viaggia identico dal mittente alla destinazione: usato come indice del nodo, ogni nodo della rete accenderebbe la stessa posizione.

L'indice sta quindi in una tabella `node_id`, scritta quando il nodo entra in servizio: dalla variabile d'ambiente `IPA_NODE_ID`, o dalla tabella nome → indice della topologia. L'indice deve seguire l'ordine dei nodi usato in addestramento, perché le decisioni significhino qualcosa sulla rete vera.

La tabella è di tipo *hash* e non *array*, e non è un dettaglio. Un array in questo ambiente è preallocato e riempito di zeri, quindi una lettura riesce **sempre**: "non configurato" e "sono il nodo 0" tornerebbero lo stesso valore. Con un hash, finché nessuno scrive non c'è voce: il vettore resta spento, ed è la risposta onesta.

7.3 La prova che contano

Stesso programma compilato, due esecuzioni che differiscono solo per la tabella del nodo:

esecuzione	esito sul pacchetto
senza indice di nodo	scartato
indice di nodo = 7	non scartato

Nient'altro cambia fra le due: l'ingresso sposta la decisione. Per l'interfaccia d'ingresso lo stesso lo mostra la consegna su veth, dove gli ingressi che raggiungono ciascuna classe dipendono dalla porta da cui il pacchetto entra.

Il prezzo sono due letture di tabella per pacchetto: P1 ne fa 6 in tutto, P3 29. In P1 la tabella dell'interfaccia costa meno istruzioni dello switch a sei casi che sostituisce: una lettura costa meno di una catena di salti.

Il verificatore paga i cammini

8.1 Che cosa fa il verificatore

Prima di accettare un programma eBPF, il kernel lo fa controllare dal **verificatore**. Il verificatore non esegue il programma su un pacchetto vero: lo percorre *su tutte le strade possibili*, per essere sicuro che nessuna faccia qualcosa di proibito. Ogni *i f* è un bivio, e il verificatore prende tutti e due i rami. Ha però un budget: **un milione di passi**. Finito il budget rifiuta il programma, per quanto corto sia:

```
BPF program is too large. Processed 1000001 insn
```

Dimensione del programma e costo di verifica sono quindi **monete diverse**. Due scelte del codice servono a spendere poco della seconda.

8.2 Il primo strato: feature fuori, neuroni dentro

Il primo strato si calcola sommando, per ogni neurone, i contributi di ogni feature dichiarata nel descrittore. L'ordine ovvio — neuroni fuori, feature dentro — chiederebbe *di che tipo* e *di che larghezza* è ogni feature una volta per neurone, cioè $8 \times 4 = 32$ volte per pacchetto, e ogni domanda è un bivio. Ma tipo e larghezza non dipendono dal neurone. Il codice di P2 e P3 gira quindi i cicli al contrario:

```
per ogni feature f (4 volte)
  di che tipo e' f?          <- 4 volte, non 32
  di che larghezza e' f?     <- 8 volte, non 64
  per ogni neurone j (8 volte)
    accumula                 <- nessun ramo qui dentro
```

Il risultato non cambia di un bit: cambia solo l'**ordine** in cui i termini si sommano, e la somma fra interi a 64 bit in complemento a due è associativa e commutativa, anche quando trabocca. È questo margine che permette a P3 di compilare il tetto sulle code a 8 e alle versioni strumentate di P2 e P3 (quelle che contano le letture di tabella) di caricarsi.

8.3 La ReLU: un bivio per neurone

Fino al 27 settembre, sulla macchina di laboratorio, due cose non si caricavano:

- **P2 con tre o più strati nascosti;**
- **P1 oltre qualche centinaio di pesi**, e prima la forma larga.

Nella macchina virtuale P2 si caricava fino a sei strati, con lo stesso sorgente: la prima spiegazione è stata "cambia il kernel". Era sbagliata. Il programma P2 a tre strati ha circa 16 000 istruzioni, e il verificatore ne percorreva un milione: stava rileggendo gli stessi pezzi di codice molte volte, una per ogni strada.

Dove stavano i bivi

Ogni neurone nascosto applica la ReLU: *se il valore è positivo tienilo, altrimenti metti zero*. Nel codice era scritta esattamente così:

```
h = a > 0 ? a : 0;      // un if: due strade
```

Un bivio per neurone. Con tre neuroni in fila le strade sono

$$2 \times 2 \times 2 = 8,$$

una per ogni combinazione di "positivo / negativo":

neurone 1	neurone 2	neurone 3	
+	+	+	strada 1
+	+	−	strada 2
+	−	+	strada 3
	...		...
−	−	−	strada 8

Con i 16 neuroni del modello largo di P1 sono $2^{16} = 65\,536$ strade. Il verificatore ha un meccanismo per riconoscere due strade "equivalenti" e percorrerne una sola (la *potatura*), ma qui riusciva a riunirne poche: i numeri qui sotto dicono che la gran parte delle strade veniva percorsa per intero.

Il verificatore scrive quanti passi ha fatto, se glielo si chiede (`diag_verifier.py`, statistiche con `log_level = 4`). Il P1 largo, 3 486 istruzioni, arrivava al milione: **287 volte** la sua lunghezza. P2 a *uno* strato era già a 799 563 passi, l'80% del budget. Il margine era così piccolo che una differenza qualsiasi fra due kernel bastava a superarlo: per questo nella macchina virtuale passava e qui no.

La ReLU senza bivio

La ReLU si può calcolare con sola aritmetica, senza if:

```
m = a >> 63;          // tutti 1 se a e' negativo, tutti 0 altrimenti
h = a & ~m;            // a se positivo, 0 se negativo
```

Lo spostamento a destra di 63 posizioni di un intero con segno a 64 bit "ricopia" il bit del segno su tutti i bit. Due esempi, mostrati su 8 bit per leggibilità (a 64 bit è identico):

	$a = 5$	$a = -3$
a in bit	00000101	11111101
$m = a \gg 63$	00000000	11111111
$\sim m$	11111111	00000000
$a \& \sim m$	00000101 = 5	00000000 = 0

Con a positivo la maschera lascia passare tutto e resta a ; con a negativo la maschera azzerava tutto. È la ReLU, **bit per bit la stessa**, ma senza bivio: il verificatore ha una strada sola da percorrere.

Il dettaglio che la fa funzionare

Scritta così e basta, non cambia niente: clang riconosce che $a \& \sim(a \gg 63)$ è "il massimo fra a e 0" e, per ottimizzare, **rimette il bivio**. Nel bytecode compilato i salti tornano tutti (136 contro 120 nel P1 largo). Serve una riga che impedisca a clang di vedere dentro la maschera:

```
static __always_inline long long ipa_relu(long long a) {
    long long m = a >> 63;
    asm volatile("" : "+r"(m));    // barriera: nessuna istruzione, m invariato
    return a & ~m;
}
```

L'asm è vuoto: non produce nessuna istruzione e non cambia m . Dice solo a clang "di m non sai niente", e clang non può più riconoscere il massimo. La funzione è la stessa in P1 (gen_full_c.py), in P2 (il leaf di ebpf_template_arch.py) e in P3 (ebpf_modular.py). P3 caricava anche prima, perché ripete un solo strato generico: la ReLU è cambiata anche lì perché le tre pipeline calcolino l'attivazione con le stesse istruzioni, e il confronto fra loro resti alla pari.

8.4 La ReLU senza bivio: il risultato

Passi del verificatore, su un budget di 1 000 000:

	ReLU con bivio	ReLU senza bivio
P2, 1 strato	799 563	120 452
P2, 2 strati	417 292	120 557
P2, 3 strati	rifiutata	127 906
P2, 6 strati	rifiutata	136 642
P2, 20 strati	rifiutata	201 528
P1 larga, 1×16 (~1 200 pesi)	rifiutata	5 698
P1 profonda, 8×9 (~1 300 pesi)	229 039	8 516
P3, layer_first	27 183	26 755
P3, layer_hidden	186 489	27 979

Adesso P2 carica fino a venti strati, e a sei ne usa il 14% del budget. P1 carica il tier da ~1 200 pesi in tutte le forme. P3 caricava già, ma lo strato che ripete (layer_hidden) costava al verificatore 117 volte la sua lunghezza: ora 17. layer_first quasi non cambia, perché la sua ReLU è una parte piccola di un programma dominato dalla lettura degli ingressi.

Il nuovo limite di P2 non è il verificatore

Oltre venti strati è **clang** a fermarsi, prima ancora del kernel:

```
LLVM ERROR: Branch target out of insn range
```

Un salto eBPF può arrivare al massimo a 32 767 istruzioni di distanza. I controlli all'inizio del programma (se il pacchetto non va bene, esci) saltano fino in fondo, e ogni strato allunga il programma di circa mille istruzioni: a venti strati il programma ne ha 32 388, appena sotto. Il terzo limite è la tabella dei pesi, 1 024 posti: 37 strati al massimo a larghezza 4, 7 a larghezza 8.

8.5 La ReLU senza bivio: che cosa costa

Il calcolo della rete è identico: i test che confrontano il C di P1 con il riferimento Python danno gli stessi logit su ogni modello sintetico (test_synth.py), e p1_c_eval.py valuta anche ipa_relu leggendone il corpo. Cambiano invece le istruzioni: circa tre per neurone al posto di due, in cambio di nessun salto. Rimisurato tutto il 28 settembre:

	baseline	P1	P2	P3
programma da solo, 27 settembre	22 ns	47 ns	199 ns	323 ns
programma da solo, 28 settembre	18 ns	52 ns	195 ns	315 ns
traffico vero, sopra la baseline, 27	—	38 ns	224 ns	368 ns
traffico vero, sopra la baseline, 28	—	38 ns	226 ns	364 ns

La baseline, che di ReLU non ne ha, è scesa di 4 ns fra i due giorni, e P2 e P3 con lei: la macchina era un po' più veloce. P1 invece è **salita** di 5 ns, quindi rispetto alla baseline la ReLU senza bivio le costa circa 9 ns nel banco a pacchetto ripetuto. Lì lo stesso pacchetto passa milioni di volte, il processore indovina sempre da che parte va ogni bivio, e un bivio indovinato è quasi gratis: toglierlo e aggiungere tre istruzioni sempre eseguite si vede. In P2 e P3 la ReLU è una parte minima di un programma molto più grande, e non si vede.

Sul traffico vero la differenza sparisce anche per P1: 38 ns sopra la baseline prima e dopo. Il prezzo è quindi piccolo e si paga soprattutto nel banco; in cambio P1 carica modelli fino a quattro volte più grandi, P2 fino a venti strati, e le tre pipeline calcolano l'attivazione con le stesse istruzioni.

"Cambia il kernel" era una spiegazione comoda e sbagliata. Il programma lavorava già all'80% del budget di verifica, e il colpevole era una riga: un `if` per neurone. Come nel capitolo precedente, il verificatore non paga le istruzioni ma le **strade**: togliere un bivio vale più che togliere cento istruzioni.

I numeri, e che cosa dicono davvero

9.1 La tabella

Modello 65-4-4-7, scala 24, `test_suite --only kernel`. P1.5 è l'oggetto AOT che va sui nodi, quello che i capitoli precedenti chiamano semplicemente P1; P1 è la specializzata, presentata nel capitolo seguente, con anche l'indice del nodo (7) compilato dentro. Misurato il 29 settembre sulla macchina di laboratorio, con la ReLU senza bivio (capitolo 8) e i contatori per-core (capitolo 12), il core del programma fisso a 3 493 MHz e isolato dal resto (capitolo 13).

	baseline	P1	P1.5	P2	P3
istruzioni caricate	135	606	1 019	15 089	12 288
salti fra programmi	0	1	1	1	3
letture di tabella	3	5	6	11	29
memoria delle tabelle	1 960 B	4 196 B	4 196 B	147 624 B	177 452 B
latenza (minimo di 7)	14 ns	48 ns	53 ns	200 ns	314 ns

Il *baseline* è un programma che analizza il pacchetto e lo rimanda fuori senza alcuna inferenza: è il pavimento contro cui si legge tutto il resto.

P1 costa 5 ns meno di P1.5: senza l'indice del nodo da leggere sparisce lo switch sui 52 nodi (413 istruzioni) e una lettura di tabella. La memoria delle tabelle comprende le copie per core dei contatori (22 CPU), per questo la baseline occupa 1 960 byte. Rispetto al 28 settembre, con i contatori condivisi, la baseline è scesa da 18 a 14 ns: i due incrementi atomici che non fa più.

Tutte le colonne vengono dalla **stessa esecuzione**, sulla stessa macchina, nello stesso minuto.

9.2 Primo risultato: la dimensione non predice la velocità

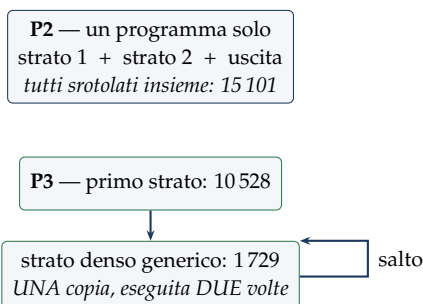
P3 ha il 19% di istruzioni in meno di P2 ed è il 57% più lento.

Il motivo sta nelle due righe centrali: tre salti fra programmi invece di uno, e ventinove letture di tabella per pacchetto invece di undici.

Il conteggio delle istruzioni misura quanto è *grande* il programma caricato, non quanto *lavoro* fa per pacchetto. In questo regime a dominare sono le letture di memoria e i salti, non l'aritmetica.

Perché P2 è più grande, pur essendo più veloce

Le due cose hanno la stessa causa: **P3 non tiene la rete intera in un programma.**



P3 srotola **un** solo strato denso generico e ci rientra, con un salto fra programmi, a ogni passo della rete. Così la *profondità* del modello non costa dimensione: un modello a sei strati gira sullo stesso identico binario. P2 deve srotolarli tutti, perché è un programma solo.

Quel riuso è esattamente ciò che P3 paga in latenza: ogni rientro è un salto più le letture che servono a ricostruire il contesto perduto. Il salto in sé costa quasi niente — misurato da solo, **un nanosecondo** — quindi quello che pesa sono le letture.

La stessa scelta di progetto spiega *entrambi* i numeri: P3 è più piccolo **perché** riusa un pezzo, ed è più lento **perché** riusarlo costa delle letture ogni volta.

9.3 Secondo risultato: quanto valgono le latenze assolute

Ogni cifra di latenza in tabella è il **minimo di sette misure** indipendenti. Guardando come sono distribuite quelle sette, dentro la singola esecuzione:

	baseline	P1	P1.5	P2	P3
minimo	14 ns	48 ns	53 ns	200 ns	314 ns
mediana	14 ns	48 ns	53 ns	200 ns	317 ns
massimo	15 ns	48 ns	54 ns	201 ns	320 ns
dispersione	7%	0%	2%	0%	2%

La dispersione sta fra lo 0 e il 7%: su 14 ns un nanosecondo è il 7%. È il risultato del condizionamento della macchina (capitolo 13): frequenza fissa e misurata, stati di sonno profondi spenti, il core del programma lasciato solo.

Le cifre assolute valgono **a quella frequenza**: il core del banco è fisso a 3,5 GHz. A un'altra frequenza cambiano quasi in proporzione; i **rapporti** fra le pipeline no.

Un caso in cui la regola è servita

Sull'asse dei nodi dell'analisi parametrica (capitolo seguente), P1.5 passa da 776 a 1718 istruzioni fra 10 e 100 nodi, e la latenza resta a 45–48 ns in tutti e cinque i punti. E il tetto sulle code di P3, portato da 1 a 8, aggiunge 3 525 istruzioni al primo strato (da 7 003 a 10 528) senza cambiare niente di ciò che gira: il modello depositato non dichiara la feature delle code, quindi quel ramo non viene mai eseguito.

Una metrica che si sposta del 50% senza che cambi nulla di ciò che gira **non è una misura di costo**. Il numero di istruzioni va letto come "quanto è grande il binario", mai come "quanto lavora".

9.4 Terzo risultato: la genericità ha un costo di verifica

P2 serve qualsiasi modello che stia sotto certi tetti dichiarati a priori (numero massimo di neuroni, di ingressi, di feature). È il suo pregio: non ricompila mai.

Il conto però lo paga il verificatore del kernel, che percorre ogni cammino del corpo srotolato. Con i tetti larghi il kernel si arrende:

```
processed 1000001 insns (limit 1000000) ... total_states 13864
```

Da notare: il programma è lungo circa 14 900 istruzioni, **ampiamente dentro** il limite di dimensione. Quello che finisce è la *complessità di verifica*, che è un limite diverso e cresce col **prodotto** dei tetti:

$$\text{costo di verifica} \sim H_{\max} \times F_{\max} \times N_{\max}.$$

Lo stesso limite fermava P2 oltre due strati nascosti, e la P1 oltre qualche centinaio di pesi, finché la ReLU non ha perso il suo bivio (capitolo 8).

È la misura concreta di quanto un programma "generico" possa esserlo. La genericità di P2 non è gratis: si paga in stati del verificatore, e il tetto si tocca con numeri ragionevoli.

9.5 Quarto risultato: togliere il compilatore dal nodo

P1 scrive i pesi *dentro* il codice come numeri letterali. È ciò che la rende veloce — il compilatore può far sparire le moltiplicazioni per zero e trasformare in scorrimenti quelle per potenze di due — ma ogni volta che il modello cambia bisogna ricompilare. Con BCC la compilazione avviene sul nodo: **75 ms** per modello, contro 2 ms di caricamento nel kernel.

L'alternativa è compilare *prima*, su una macchina qualunque, e spedire al nodo l'oggetto già pronto. Al momento del deploy il nodo apre il file e lo carica, senza compilatore:

oggetto precompilato (AOT)	
compilazione, una volta, fuori dal nodo	75,8 ms
messa in servizio sul nodo (apri, verifica, JIT)	1,12 ms
istruzioni	1 019
latenza per pacchetto	51–53 ns

Il nodo non ha più bisogno di un compilatore, e il costo per pacchetto è lo stesso di tutte le altre misure: **ogni cifra di P1 in questo quaderno è dell'oggetto AOT**. La riduzione fatta dal compilatore sui pesi letterali resta dentro l'oggetto.

Vale solo quando i modelli sono noti in anticipo, che è l'ipotesi sotto cui P1 esiste.

9.6 Che cosa questi numeri non dicono

BPF_PROG_TEST_RUN esegue il programma in un ciclo sullo stesso buffer. Niente scheda di rete, niente driver, nessuna allocazione, nessuna pressione di cache da traffico vero.

Il valore in "milioni di pacchetti al secondo" è semplicemente $1/\text{latenza}$: un **picco teorico**, non un throughput retto. Per quello serve traffico vero, generato a ritmo, e la misura di quanti pacchetti si perdono (capitolo 12).

Come cresce il costo

10.1 La domanda

Fin qui tutto è misurato su *un* modello e *una* topologia. La domanda che manca è come cambia il costo quando la rete o il modello diventano più grandi — e soprattutto se le pipeline hanno **pendenze diverse**.

Se le pendenze fossero uguali, averne più di una sarebbe un esercizio. Non lo sono, ed è questo il capitolo che lo mostra.

Le pipeline confrontate sono **quattro**, e formano una scala di *quanto si sa quando il programma viene compilato*:

	compilato dentro	un binario per
P1 specializzata	pesi <i>e</i> indice di questo nodo	nodo
P1.5 hardcoded	pesi; il nodo si legge a runtime	modello
P2 template	solo i soffitti	tutto
P3 modular	anche la profondità è a runtime	tutto

La P1.5 è quella che i capitoli precedenti chiamano semplicemente P1: la *disposizione* delle feature è scritta nel codice, ma l'indice del nodo no. La P1 specializzata congela anche quello.

10.2 Come è impostata la misura

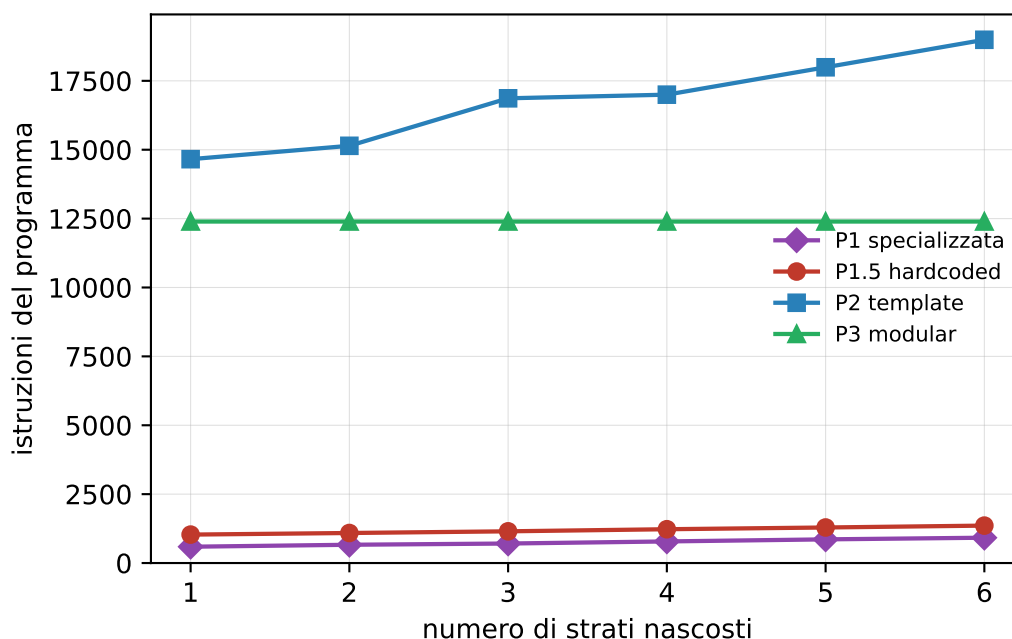
Sette assi, e su ciascuno si muove **una sola** variabile: tutto il resto resta fermo, così una curva si può attribuire a quello che c'è scritto sull'asse delle ascisse e a nient'altro.

asse	valori	si ferma perché
nodi della rete	10, 25, 52, 75, 100	il tetto sugli ingressi è 128
hidden layer	da 1 a 6	—
a parametri fermi	da 1 a 5 strati, 592 pesi	—
neuroni per layer	2, 4, 6, 8	il tetto compilato è 8
composizione dell'IV	4 varianti	0, 1 o 2 one-hot
pesi a zero	0% ... 90%	—
porte presenti	da 2 a 6	—

Tre accortezze che cambiano i risultati, non solo la forma:

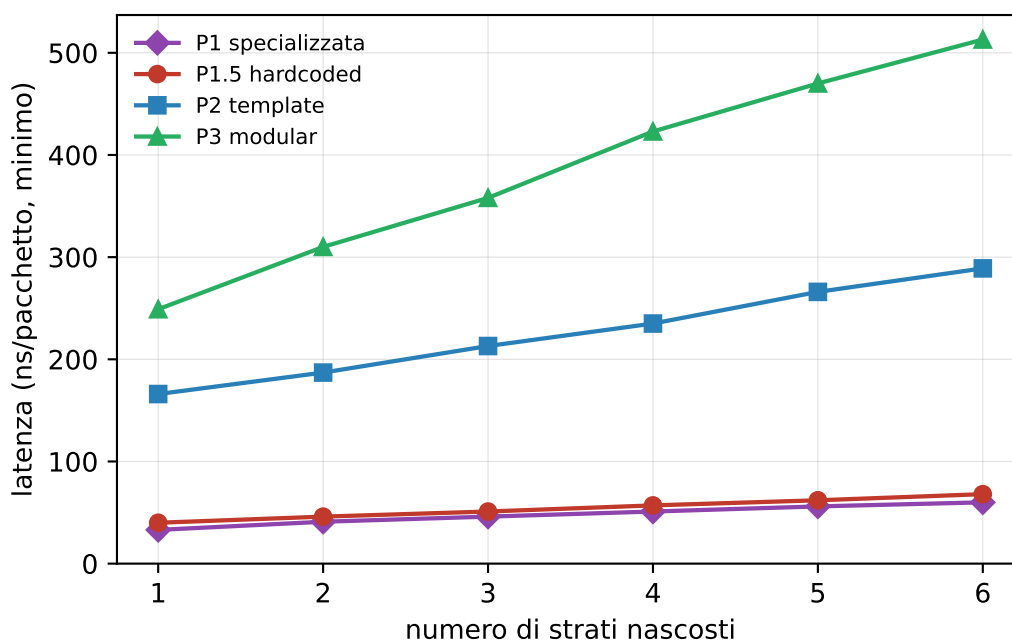
- **Ogni misura in un processo separato.** P1 può far abortire il compilatore in modo non recuperabile; e un rifiuto del verificatore è un *dato*, non un guasto dello strumento.
- **I pesi vengono da un sacchetto fisso**, e un modello più grande *estende* quello più piccolo invece di sostituirne tutti i pesi: in P1 il numero di istruzioni dipende dai valori dei pesi, e cambiarli tutti a ogni punto mescolerebbe il loro effetto con quello dell'asse.
- **Un allarme automatico sulle misure sporche:** se una pipeline ha lo stesso identico programma e la latenza varia più del doppio lungo l'asse, lo scarto è la macchina e lo strumento lo dice.

10.3 Profondità: chi paga in spazio e chi in tempo



P3 ha la stessa dimensione da uno a sei strati: 12 394 istruzioni in ogni punto. **P2 cresce** di circa 870 istruzioni per strato, da 14 657 a 18 990.

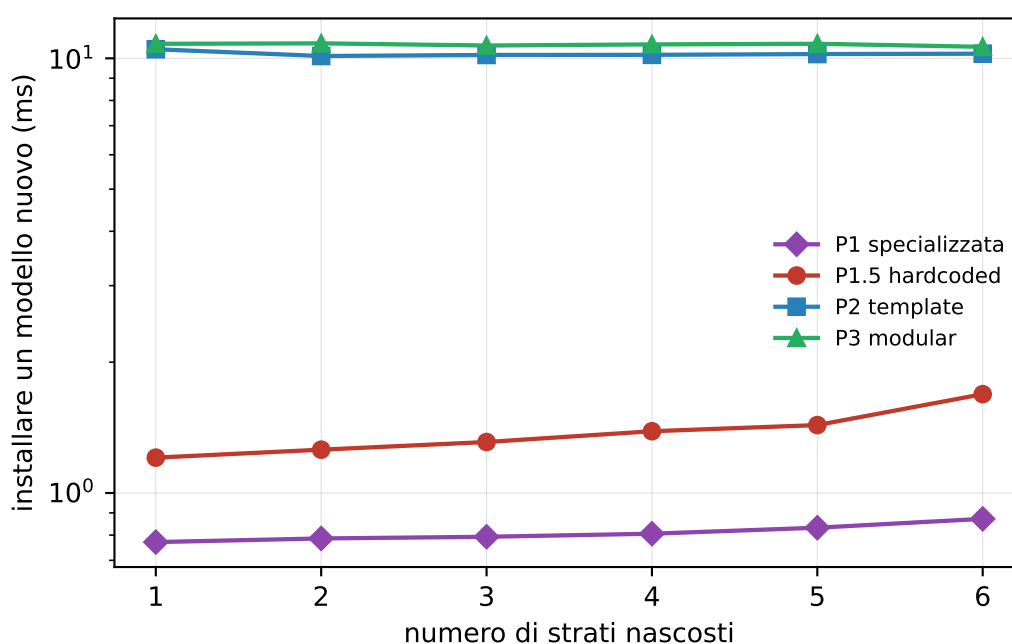
P3 srotola *un* solo strato denso generico e ci rientra a ogni passo della rete, quindi la profondità non entra nel programma. P2 deve srotolarli tutti, perché è un programma solo. Fino al 27 settembre P2 oltre due strati non si caricava: il verificatore si arrendeva, per colpa di un bivio in ogni ReLU (capitolo 8). Questa è la prima misura con tutti e sei i punti.



E P3 lo paga in tempo: circa 53 nanosecondi per strato (249 → 513 ns da uno a sei strati), a istruzioni identiche. **P2 ne paga circa 25** (166 → 289 ns): lo strato in più è codice in linea, niente salti e niente letture per ricostruire il contesto. Le due P1 crescono di circa 5 ns per strato (33 → 60 la specializzata, 40 → 68 la P1.5).

Le due figure hanno la *stessa* causa vista dai due lati. P3 è più piccolo **perché** riusa un pezzo, ed è più lento **perché** riusarlo costa un salto e delle letture ogni volta. È il compromesso della modularità, ridotto a due numeri. La differenza vera sulla profondità resta un'altra: P3 cambia profondità senza ricompilare, P2 deve ricompilare il programma.

10.4 Aggiornare il modello: con l'oggetto precompilato, pari



Con P1 costruita come oggetto precompilato, installare un modello nuovo su un nodo già in servizio costa **0,7–2,1 millisecondi** in P1 — il nodo apre e carica un file pronto — e **10–12 millisecondi** in P2 e P3, che scrivono in una tabella (pesi e semantica delle classi).

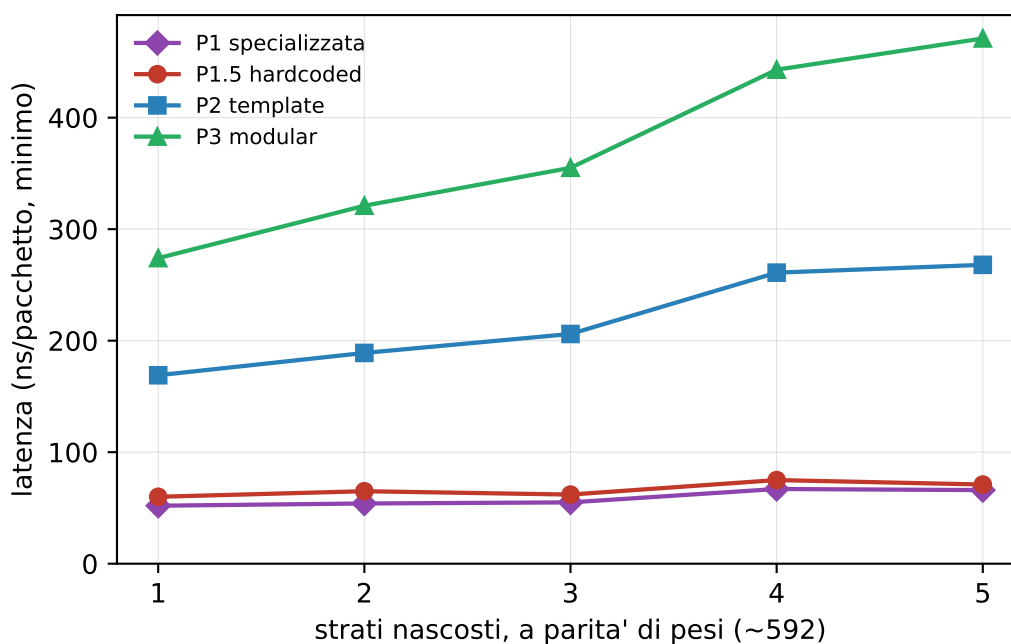
Contando anche il lavoro di P1 fuori dal nodo — generare il C, compilarlo con clang — si sta fra 66 e 120 ms. P2 e P3 compilano con BCC una volta sola, all'avvio del nodo (1,2–1,7 s), e mai più.

La differenza che resta fra le pipeline non è quindi il tempo di aggiornamento, ma *dove* serve il compilatore: P1 ne vuole uno per ogni modello nuovo, su qualche macchina; P2 e P3 no.

10.5 Larga o profonda? Dipende da chi la esegue

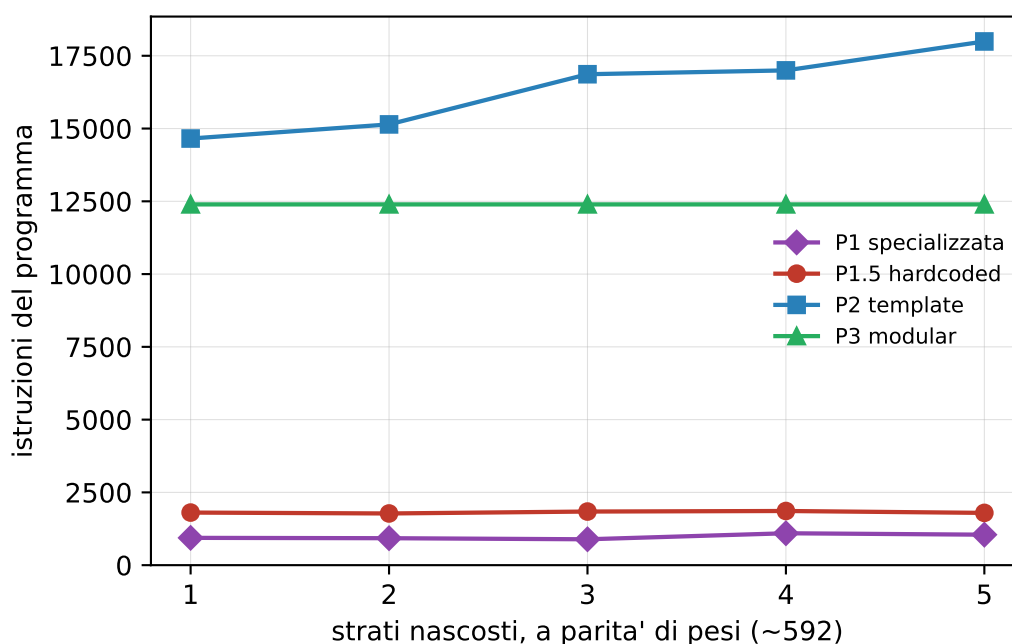
Dato un numero di parametri da spendere, conviene metterli in *un* layer largo o in *molte* stretti? È una domanda che si fa di solito parlando del modello, come se la risposta fosse una proprietà della rete neurale. Qui non lo è.

Cinque forme, tutte con lo stesso budget — 592 pesi, con uno scarto del 2% — disposto in modo diverso: da un solo strato da otto neuroni a cinque strati stretti.



A parità di parametri le due versioni hardcoded cambiano poco e non in modo monotono (52–67 ns la specializzata, 60–75 la P1.5: circa +25% fra gli estremi). P2 perde il 59% da uno a cinque strati (169 → 268 ns), P3 il 72% (274 → 471 ns).

La stessa scelta di architettura costa poco o costa molto, a seconda della pipeline su cui finisce.



P3 ha la riga **perfettamente orizzontale** — 12 394 istruzioni a tutti e cinque i punti — e perde comunque il 72% di velocità.

Dimensione costante e tempo crescente è la firma di un costo di transizione, non di calcolo. In P3 si vede allo stato puro: ciò che cresce non è il lavoro, è il numero di volte che il programma deve rientrare in sé stesso.

Con budget diversi, e ingressi diversi

Il banco dedicato (bench_depth_vs_width) prova tre budget e quattro vettori d'ingresso, sulla sola P1:

	larga	2 strati	4 strati	8 strati	
<i>~300 pesi</i>					
ingresso con due one-hot (65 valori)	39 ns	46	49	64	(+64%)
ingresso denso (11 valori)	93 ns	88	83	81	(−13%)
<i>~1 200 pesi</i>					
ingresso con due one-hot (65 valori)	110 ns	150	176	219	(+99%)
ingresso denso (11 valori)	stack	stack	320	291	(−9%)

P1 come oggetto AOT, ReLU senza bivio, 28 settembre. *stack*: clang non compila, la forma chiede più dei 512 byte di stack di eBPF.

Con un ingresso **grande e dominato da una one-hot** — il caso di IPA, 52 dei 65 valori — allargare batte approfondire, e a 1 200 pesi la rete a otto strati costa il doppio di quella larga. Con un ingresso **piccolo e denso** vince la rete profonda: più veloce del 13% a 300 pesi e anche più piccola (1 368 contro 1 535 istruzioni). Il motivo: una one-hot larga costa poco per pacchetto, perché il datapath ne legge *una colonna sola*; un ingresso denso invece fa moltiplicare le letture per la larghezza del primo strato.

I limiti di P1, prima e dopo il 28 settembre

Fino al 27 settembre, a $\sim 1\,200$ pesi sull'ingresso di IPA **caricava solo la forma a otto strati**: la larga, quella a due e quella a quattro strati venivano rifiutate dal verificatore (Processed 1000001 insn). La causa era un bivio in ogni ReLU (capitolo 8), e la forma larga ne ha di più in fila. Con la ReLU senza bivio carica tutto il tier.

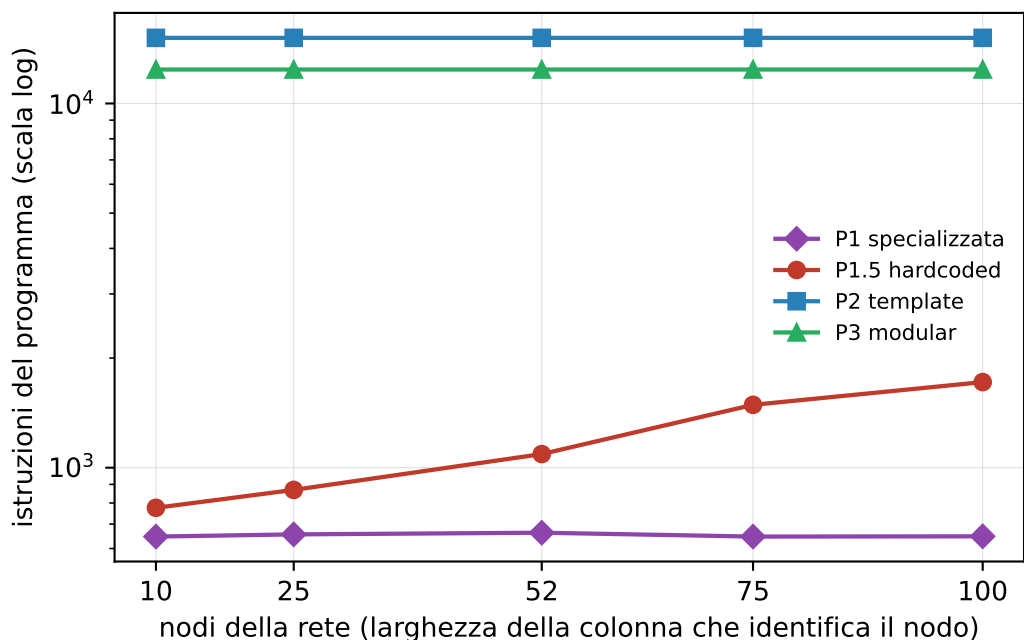
Il limite che resta è un altro, e non è del verificatore: lo **stack** di eBPF, 512 byte. Con l'ingresso denso a 1 200 pesi la forma larga (un solo strato da 63 neuroni) e quella a due strati da 26 chiedono più spazio per i valori intermedi di quanto il programma ne abbia, e clang si ferma prima di produrre l'oggetto. A $\sim 4\,700$ pesi succede a tutte le forme.

La frase giusta non è "le reti larghe sono preferibili a quelle profonde". È: *con un ingresso dominato da feature one-hot, allargare batte approfondire; con un ingresso piccolo e denso vince la rete profonda; e oltre qualche centinaio di pesi, in P1, è la forma larga a esaurire per prima lo stack.*

10.6 Congelare il nodo: conviene?

L'indice del nodo è una **costante del deployment**: un nodo non cambia di posto. Il codice però lo legge a runtime, e per farlo porta uno switch con un caso per ogni nodo della rete — 52 casi per Germany50, di cui ne esegue *uno*.

Congelandolo a tempo di generazione lo switch sparisce, e con lui la lettura della tabella. Restano quattro numeri costanti, che il compilatore può piegare dentro al conto.



La taglia della rete entra nel programma **solo** nella P1.5, che passa da 776 a 1718 istruzioni fra 10 e 100 nodi.

Congelando il nodo quella dipendenza **scompare**: la riga della specializzata è piatta, fra 647 e 663 a qualunque taglia.

(La scala è logaritmica: in lineare le due righe P1 resterebbero schiacciate contro le quindicimila di P2.)

E la velocità

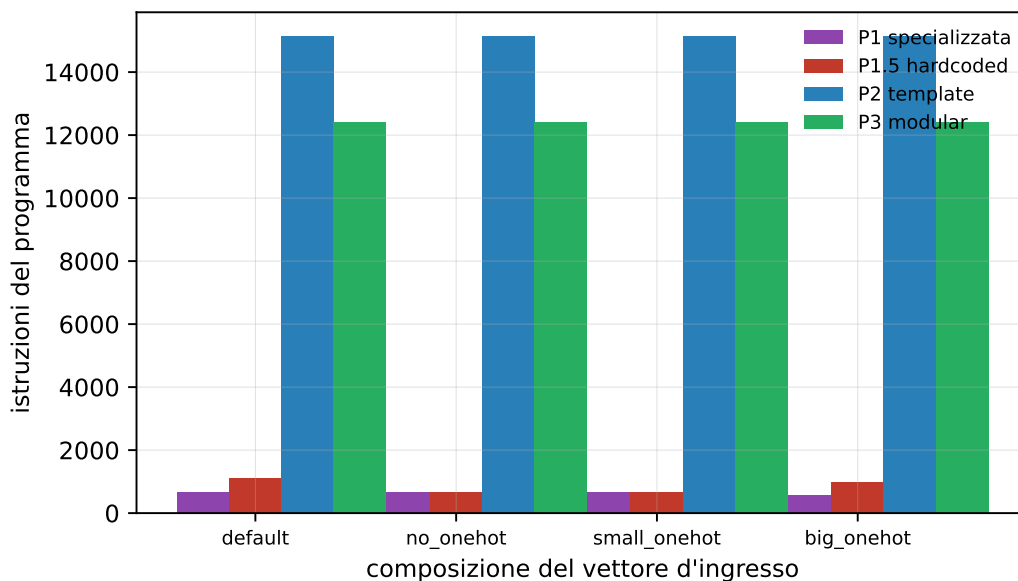
nodi	10	25	52	75	100
P1 specializzata	41	40	41	41	38 ns
P1.5 hardcoded	48	45	46	47	45 ns

La specializzata è più veloce di circa 6 ns, il 13%, a ogni taglia: la lettura della tabella del nodo e lo switch costano poco, ma si vedono. Entrambe sono piatte: a runtime lo switch è un salto indicizzato, qualunque sia il numero dei casi.

Il motivo per cui la *dimensione* crolla e il *tempo* cala poco è lo stesso: lo switch ha 52 casi ma ne esegue uno. Quelle 208 assegnazioni sono compilate per eseguirne quattro. Toglierle libera molto **spazio** e un po' di **tempo**.

Il controllo, e perché ci si può fidare

Come si fa a essere sicuri che il divario venga davvero dalla one-hot del nodo e non da qualcos'altro? Si prende un vettore d'ingresso che la feature *non* la contiene:



Nelle due composizioni senza la feature node le due P1 sono **identiche alla cifra** (656 e 656; 649 e 649). In quelle che ce l'hanno, divergono. Il divario è tutto lì e nient'altro.

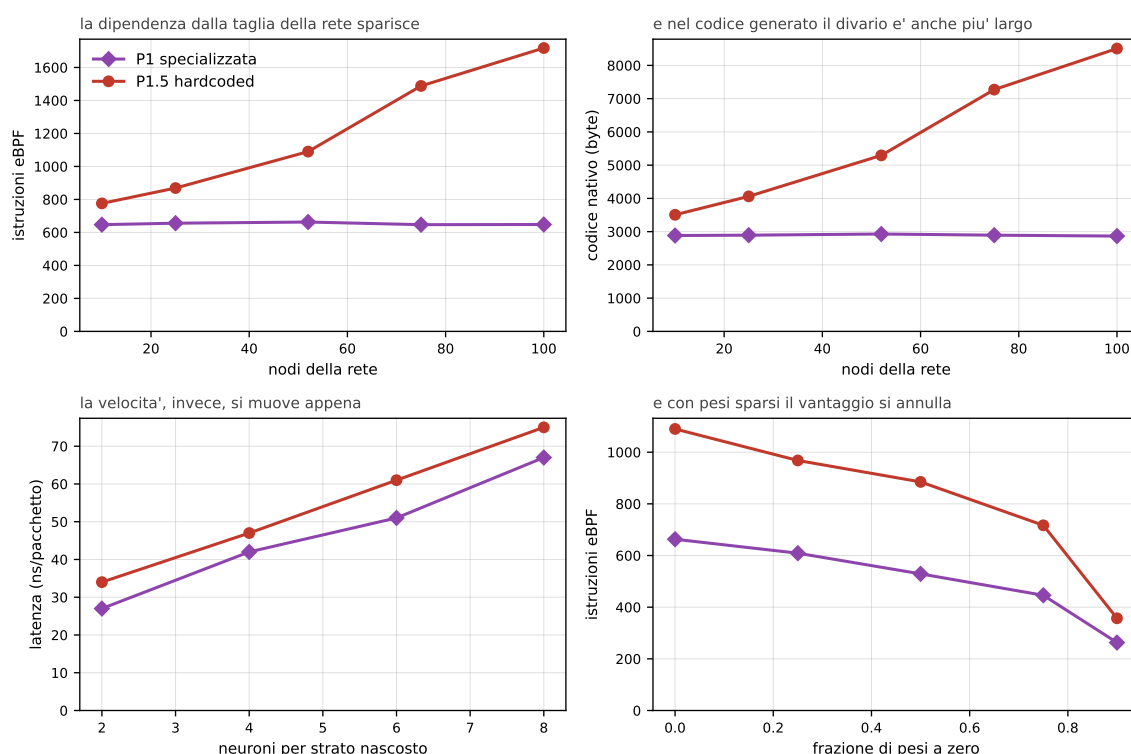
C'è poi una verifica che viene prima di ogni numero. Congelare il nodo significa scegliere *una colonna* della matrice dei pesi al momento di generare il codice: sceglierne una sbagliata darebbe un programma più piccolo, più veloce, e che calcola **un altro modello**. Ottanta casi, dieci valori di TTL per otto configurazioni di link: la versione congelata decide sempre la stessa classe di quella che legge il nodo dalla tabella.

I due vantaggi non si sommano

pesi a zero	0%	25%	50%	75%	90%
P1 specializzata	663	609	529	446	263
P1.5 hardcoded	1 090	968	885	717	357

Al 90% di zeri il divario scende da 427 a 94 istruzioni. Sparsità e nodo congelato sono **due strade alla stessa riduzione**: se i pesi sono già quasi tutti zero, il compilatore cancella lo switch da solo e non resta niente da congelare.

Le quattro risposte in una figura



Qui ci sono solo le due P1: in tutte le altre figure P2 e P3 stanno un ordine di grandezza più su e schiacciano questo confronto sul fondo del grafico. La dimensione crolla e smette di dipendere dalla rete; il codice nativo crolla ancora di più (2867 byte contro 8507 a cento nodi); la velocità migliora di poco; e con i pesi sparsi le due curve si toccano.

Il verdetto

Congelare il nodo **non è soprattutto un'ottimizzazione di velocità**. È ciò che rende la pipeline hardcoded *praticabile su una rete grande*.

Lo switch della P1.5 cresce col numero di nodi moltiplicato per la larghezza del primo strato. Su una topologia da 500 nodi con 8 neuroni sarebbero quattromila assegnazioni srotolate, e c'è una taglia oltre la quale quel programma non si carica più. La specializzata non ci arriva mai.

Il prezzo è **un binario per nodo**: 52 compilazioni (circa 70 ms ciascuna) e 52 installazioni.

10.7 Congelare quali porte esistono

La sezione precedente congela *dove* il nodo si trova. Questa congela *quante* interfacce ha.

Il modello riserva sei colonne per `link_state` perché sei è il grado del nodo più connesso della rete. Ma su un nodo di grado 3 le tre colonne restanti non sono link caduti: dietro non c'è alcuna interfaccia. Valgono zero adesso, varranno zero sempre.

Non si congela il **valore** della feature, si congela la sua **struttura**. Se un link sia su o giù resta letto a runtime; quello che diventa noto alla compilazione è *quali posizioni possono esistere*.

Il generatore non emette le colonne che restano fuori. Il termine che sparisce è `link_state[i] · w[j][i]` con `link_state[i]` nullo: la somma è fra interi a 64 bit, quindi togliere un addendo nullo non cambia un bit.

porte presenti	2	3	4	5	6
istruzioni eBPF	579	599	615	639	663
latenza	37	38	39	39	41 ns

21 istruzioni per porta, in modo lineare, e **4 ns** fra grado 6 e grado 2, con la latenza che sale in modo monotono. Diversamente dal nodo, qui le moltiplicazioni tolte vengono eseguite davvero a ogni pacchetto, e la misura le vede: circa un nanosecondo per porta.

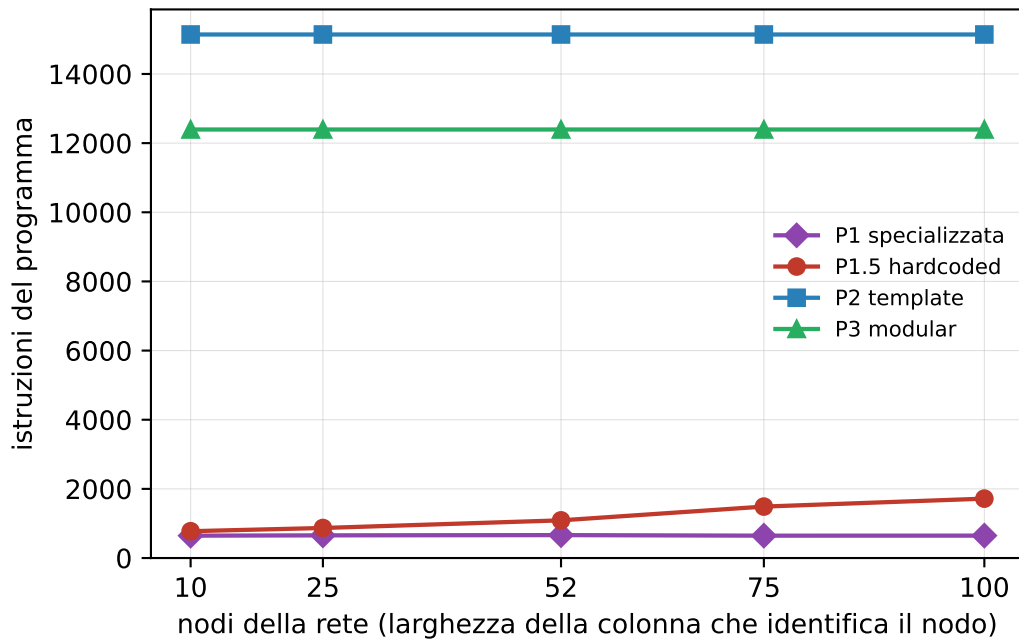
Le altre tre pipeline, che non specializzano niente, restano ferme lungo l'asse (1 090, 15 142 e 12 394 istruzioni, latenze piatte): la pendenza della specializzata viene dalle porte e da nient'altro.

Il controllo, che è servito davvero

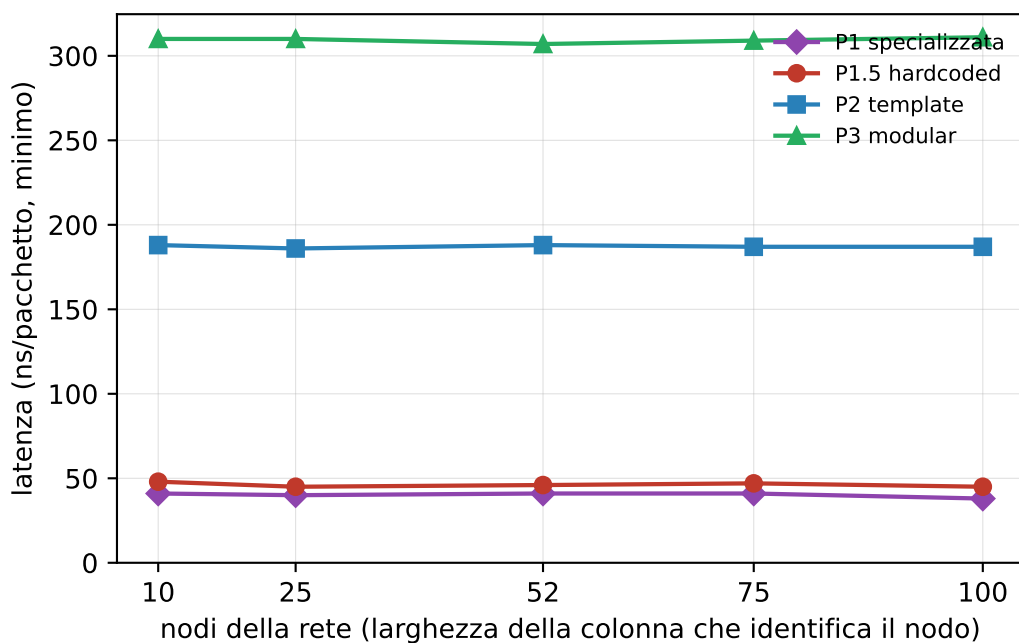
Ottanta casi — dieci valori di TTL per otto configurazioni di link realizzabili — danno la stessa classe fra la versione specializzata e quella a larghezza piena. Ma quel numero, da solo, non dimostra niente: se quelle colonne non spostassero mai la decisione, «le due versioni concordano» sarebbe vero anche per una specializzazione **sbagliata**. Serve un controllo al contrario: accendere uno slot che sul nodo non esiste, e pretendere che a quel punto le due divergano. Su alcuni pesi quel controllo resta muto — una colonna contribuisce $1 · w$ a un accumulatore che ne somma 65 — e il banco cerca fra più semi finché non morde: 29 divergenze.

Un test che passa anche quando la sua ipotesi è violata non sta verificando l'ipotesi.

10.8 Due larghezze, due risposte opposte

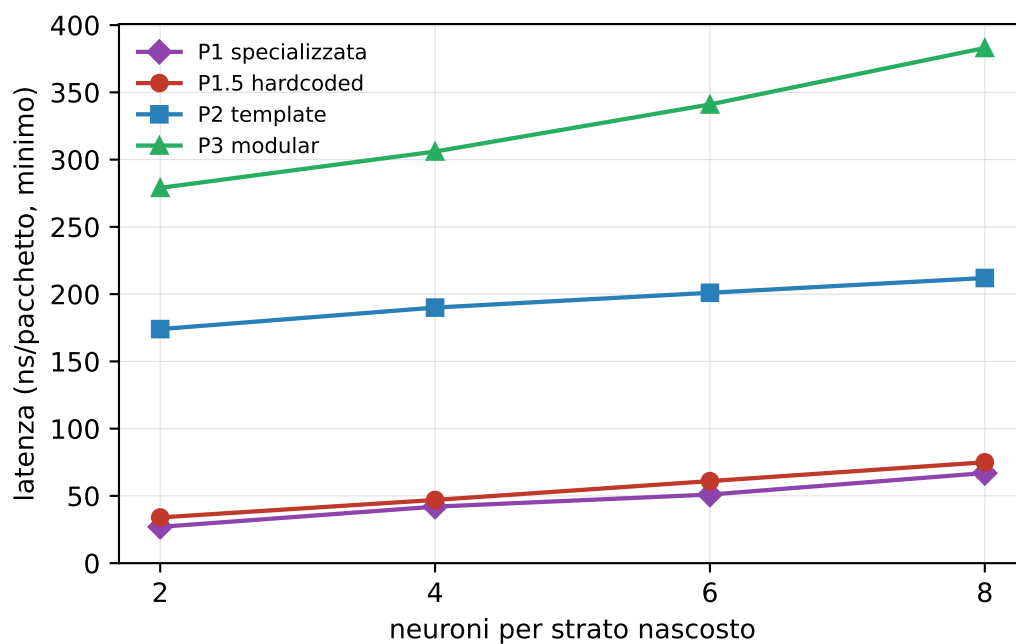


La taglia della *rete* entra nel programma solo in P1.5: è lo switch della one-hot che si srotola. P2 e P3 restano alla cifra esatta a ogni punto.



Piatte tutte. Cento nodi costano quanto dieci: una one-hot ha un solo uno, quindi il datapath legge *una sola colonna di pesi* qualunque sia la larghezza del vettore.

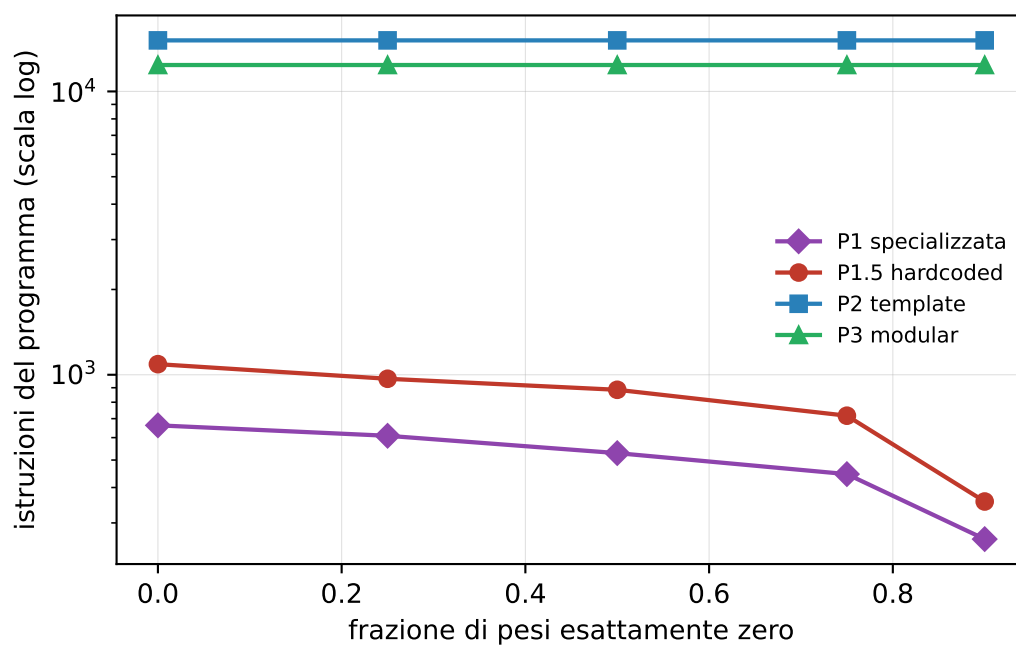
Allargare i layer *nascosti*, invece:



Da 2 a 8 neuroni: P1 specializzata 27 → 67 ns, P1.5 34 → 75, P2 174 → 212, P3 279 → 383.

La rete può crescere quanto vuole; il modello no.

10.9 I pesi contano, ma solo per una delle tre



Con il 90% dei pesi a zero, P1.5 passa da 1 090 a **357 istruzioni** e da 47 a 26 ns. P2 e P3 non si muovono di una istruzione, e di un paio di nanosecondi al massimo.

P1 scrive i pesi nel codice come numeri, quindi il compilatore vede una moltiplicazione per zero e la cancella. Per P2 e P3 uno zero è un byte in una tabella come un altro. È l'unico asse su cui dimensione e tempo scendono insieme.

10.10 Che cosa questi grafici non dicono

- Sono tutti BPF_PROG_TEST_RUN: il programma, non il nodo sotto traffico (capitolo 12).
- Oltre circa 115 nodi non si va senza alzare un tetto compilato e **rimisurare**.
- Che i *pesi*, a forma fissa, possano decidere se P1 si carica è possibile (il codice generato dipende dai valori), ma variando il solo seme dei pesi non è stato misurato. Con la ReLU senza bivio il verificatore ha molto margine (capitolo 8), quindi è meno probabile di prima.

Il costo del programma, misurato da solo

Prima di mettere un programma in una rete conviene sapere quanto costa da solo. Qui lo strumento è `BPF_PROG_TEST_RUN`: il kernel esegue il programma su un pacchetto preparato, in un ciclo, senza schede di rete, code o altri programmi di mezzo. È come cronometrare un cassiere mentre batte sempre lo stesso scontrino: misura il gesto, non la fila. La fila è l'argomento del capitolo 12.

11.1 La campagna sulle architetture

Per rispondere a *come cresce il costo quando il modello cresce* servono più modelli misurati con la stessa metodologia: quattro assi, un solo parametro che si muove per asse, minimo su sette prove.

Asse	Valori	Che cosa muove
<code>iv_dense</code>	n_{in} 5, 9, 13, 17	colonne d'ingresso tutte <i>dense</i> : ognuna è letta da una mappa e moltiplicata
<code>iv_onehot</code>	n_{in} 16, 32, 65	colonne d'ingresso in una <i>one-hot</i>
<code>width_camp</code>	65- $v-v-7$, v 4, 8, 16, 32	neuroni per strato nascosto
<code>depth_camp</code>	65-8...8-7, 1-4 strati	profondità, a larghezza fissa 8

I muri, tutti dichiarati

Larghezza 16 e 32 su Pipeline 2 e 3. I soffitti compilati valgono 8. E Pipeline 3 non rifiuta il caricamento: risponde `XDP_PASS` a runtime. Una cella a larghezza 16 *si caricherebbe, girerebbe, e restituirebbe una latenza* — di un programma che non calcola niente. Il banco controlla i soffitti *prima* di compilare e segna la cella RIFIUTATO.

Larghezza 32 su Pipeline 1. La compilazione con clang fallisce (lo stack di eBPF non basta). Ogni cella gira nel proprio sottoprocesso: la cella esce CRASH e lo sweep continua. Fino al 27 settembre anche larghezza 16 veniva rifiutata, dal verificatore, e P2 oltre due strati: con la ReLU senza bivio (capitolo 8) P1 a larghezza 16 carica (166–170 ns) e P2 carica tutti i punti di `depth_camp`.

11.2 Il problema: le MAC nominali non predicono la latenza

Il modo naturale di mettere in relazione complessità e tempo è contare le moltiplicazioni-accumulo dalla forma del modello: $n_{in} \times h_1$ per il primo strato, e così via. Fatto così, non funziona: con le MAC nominali sull'asse delle ascisse la retta spiega il 62% della varianza per la P1 specializzata, il 22% per P2 e l'8% per P3.

Il sintomo più chiaro sta sull'asse `iv_onehot`, dove il vettore d'ingresso quadruplica:

n_{in}	pesi	p1_static	hardcoded	template	modular
16	271	65 ns	65 ns	170 ns	363 ns
32	399	65 ns	64 ns	170 ns	367 ns
65	663	64 ns	64 ns	169 ns	367 ns

n_{in} per quattro, pesi per 2,4, e la latenza **non si muove su nessuna delle quattro**. Le istruzioni di Pipeline 1 intanto salgono da 1144 a 1632: la taglia statica cresce, il percorso eseguito no.

11.3 La soluzione: contare le MAC che vengono eseguite

La causa è nel datapath, ed è una riga sola. Il ramo che tratta una feature one-hot è questo:

```

} else if (code == FEAT_INGRESS_IF) {
    if (_raw_iface >= 1 && _raw_iface <= sz) {
        __u32 c = coff + (_raw_iface - 1);
        for (int j = 0; j < ML1_MAX_H1; j++)
            out[j] += LW_W(LW, woff + j * n_in + c);
    }
}

```

Una one-hot occupa sz colonne nella matrice dei pesi, ma nel datapath ne attiva *una*: il costo è h_1 addizioni, e non dipende da sz . Contarla come $sz \times h_1$ moltiplicazioni sovrastima, e di quanto dipende da quanta one-hot c'è nell'ingresso — cioè cambia da asse ad asse.

Il banco calcola quindi due colonne: `macs`, nominali, e `macs_eff`, che conta le one-hot per quello che eseguono. Con la seconda sull'asse delle ascisse:

Pipeline	MAC eseguite		MAC nominali	
	ns/MAC	r^2	ns/MAC	r^2
p1_static	0,287	0,99	0,070	0,62
hardcoded	0,290	0,98	0,076	0,70
template	0,548	0,71	0,085	0,22
modular	1,175	0,91	0,098	0,08

Per Pipeline 3 si passa da $r^2 = 0,08$ a $r^2 = 0,91$, e `p1_static` da 0,62 a 0,99. Quattro assi costruiti in modi diversi collassano sulla stessa retta, una per pipeline. P2 si ferma a $r^2 = 0,71$: ha un costo fisso grande rispetto alla pendenza, e il suo residuo più grande sta sull'asse della larghezza, dove i neuroni oltre la larghezza del modello vengono calcolati comunque fino al soffitto compilato.

Il costo per MAC eseguita è una proprietà misurabile della pipeline: circa 0,29 ns quando i pesi sono compilati come letterali, circa 1,18 ns quando vengono letti da una tabella (P3). Un fattore 4,1×: il prezzo della genericità espresso in una costante invece che in un aneddoto.

Il pavimento della baseline, che di MAC ne esegue zero, sta fra 18 e 23 ns: è il costo del framework XDP che ogni pipeline paga prima di moltiplicare qualunque cosa.

11.4 Due banchi, lo stesso programma, lo stesso numero

I due banchi a `BPF_PROG_TEST_RUN` misurano lo stesso programma, e devono dare lo stesso numero. Sul modello 65-4-4-7:

Pipeline	test_suite	campagna	scarto
baseline	18 ns	18 ns	0%
hardcoded	52 ns	46 ns	−12%
template	195 ns	190 ns	−3%
modular	315 ns	305 ns	−3%

Concordano entro qualche nanosecondo, e i programmi non sono identici: `test_suite` usa i pesi veri del modello (1 064 istruzioni per P1), la campagna pesi sintetici a sparsità nulla (1 090).

Il dettaglio che lo rende vero: un pacchetto nuovo ogni 200 giri

Il kernel, dentro `BPF_PROG_TEST_RUN`, non ripristina il pacchetto fra una ripetizione e l'altra. Il datapath decrementa il TTL: riusando lo stesso pacchetto, dopo una quarantina di esecuzioni il TTL resterebbe inchiodato a 1, e da lì in poi ogni ripetizione prenderebbe il ramo

```
if (ip->ttl <= 1) { /* ... */ return XDP_PASS; }
```

che sta **dopo** l'inferenza — il modello girerebbe comunque, e le curve scalerebbero in modo regolare — ma salta la coda di inoltrare: decremento, checksum, contatori, `mac_table`, `bpf_redirect`. Tutti i banchi rinfrescano quindi il pacchetto ogni 200 esecuzioni partendo da TTL 255 ($255 - 200 = 55$): nessuna ripetizione arriva alla scadenza.

Un difetto che lascia il modello girare è il più difficile da vedere: le curve restano belle. Si vede solo confrontando due banchi che devono dare lo stesso numero.

11.5 Il programma sul percorso vero: tre marcature

Con traffico vero si può cronometrare il programma *dentro* il percorso: il dispatcher marca T_1 all'ingresso, il programma marca T_2 subito prima di inoltrare, il contatore all'altro capo marca T_3 .

	baseline	P1	P1.5	P2	P3
$T_2 - T_1$ (il programma)	34 ns	68	77	228	340
$T_3 - T_2$ (redirect, veth, ricezione)	194–239	195–244	195–251	200–257	207–251

$T_2 - T_1$ è piatto sul carico, da 0,5 a 3 milioni di pacchetti al secondo; $T_3 - T_2$ è un costo comune di circa 195–255 ns che non dipende dalla pipeline. Sopra la baseline il solo programma costa +34 / +43 / +194 / +306 ns. La versione strumentata aggiunge due letture dell'orologio per pacchetto: se qualcosa, questi numeri sono gonfiati.

Il test reale: quanto costa un pacchetto al nodo

12.1 La domanda, in parole semplici

Un nodo della rete riceve pacchetti, per ognuno esegue la rete neurale, decide da quale porta farlo uscire, e lo spedisce. La domanda è: **quanti pacchetti al secondo regge il nodo, e quanto tempo spende su ognuno?**

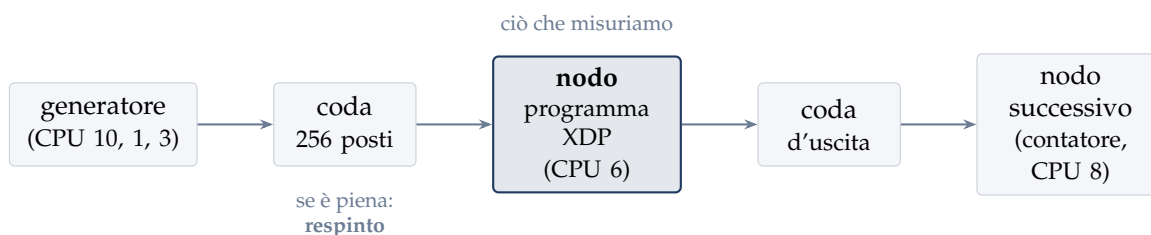
Un'immagine per tutto il capitolo: la cassa del supermercato

Il nodo è un **cassiere**. I pacchetti sono i **clienti**. Il generatore di traffico è la porta da cui entrano i clienti, e il collegamento virtuale fra generatore e nodo è il **nastro** su cui i clienti aspettano: ha 256 posti, e quando è pieno chi arriva viene mandato via.

Per sapere quanto spende il cassiere per cliente si fanno arrivare più clienti di quanti ne riesca a servire, così che il nastro non sia mai vuoto, e si conta quanti ne serve in un secondo. Se ne serve 5 milioni al secondo, spende $1/5\,000\,000$ di secondo a cliente, cioè 200 nanosecondi.

12.2 Com'è fatto il banco

Tutto sta sulla macchina di laboratorio, e ogni parte ha il suo core fisico, a frequenza fissa (capitolo 13):



- Il **generatore** (pktgen, nel kernel) crea pacchetti uguali a quelli veri: stesso formato, stesso header IPA, stesso modello da eseguire. Tre thread, tutti sulla stessa coda del nodo.
- La **coda d'ingresso** è quella di un *veth*, il cavo virtuale di Linux: 256 posti, e la dimensione non si può cambiare.
- Il **nodo** è una delle pipeline caricata in XDP. È l'unica cosa che si vuole misurare.
- Il **nodo successivo** conta i pacchetti arrivati e li butta.

Oltre alle cinque pipeline il banco misura, nella stessa sessione, *rxonly*: un programma che prende il pacchetto dalla coda e lo butta senza guardarlo. Dice quanto costa *soltanto ricevere*: è il pavimento sotto il quale nessuna pipeline può scendere.

12.3 Come si misura, e perché così

Un numero affidabile richiede cinque accortezze. Nessuna riguarda le pipeline: senza, tutte farebbero sembrare il nodo diverso da com'è.

1. La finestra stazionaria

`pktgen` si avvia con un comando che *blocca* finché la trasmissione non è finita, e il blocco non dura quanto la trasmissione: aspetta sempre 125 millisecondi fissi prima di partire, i suoi thread si accorgono del via con fino a 100 ms di ritardo, e alla fine controlla ogni 100 ms se hanno finito. Dividere i pacchetti per la durata del blocco vorrebbe dire contare fino a circa **225 ms di tempo morto** su finestre di misura da 300 ms.

Il banco aspetta che tutti i thread del generatore stiano trasmettendo, lascia assestare la coda, legge i contatori, aspetta 300 ms, li rilegge, e solo allora ferma il generatore. Il rate è la differenza fra le due letture divisa per il tempo fra le due letture.

2. Letture brevi

Ogni lettura (generatore più nodo) deve durare meno di 3 ms; se dura di più viene rifatta, e le si attribuisce l'istante a metà. Un processo interrotto a metà lettura attribuirebbe ai contatori un istante sbagliato di tutta l'interruzione. Un test senza kernel lo verifica con un generatore simulato.

3. I respinti contano

Quando la coda d'ingresso è piena il collegamento rifiuta il pacchetto («respinto»). Un pacchetto respinto è un pacchetto che il nodo non ha preso, perché non svuotava la coda abbastanza in fretta: è **una perdita del nodo**. Il banco conta separatamente i **respinti** (mai entrati) e i **persi dopo l'ingresso** (entrati ma non usciti).

4. La copia del generatore

`pktgen` crea i pacchetti come li crea lo stack di rete di Linux, in una *skb* con 64 byte liberi davanti; XDP ne vuole 256. Per ogni pacchetto il collegamento virtuale lo **copia** prima di darlo al nodo, e la copia la paga il processore del nodo. Su una scheda di rete con XDP nativo non esiste. Per toglierla c'è un secondo generatore (`ipa/test/xdp_gen.py`), che crea pacchetti già grezzi con `BPF_PROG_TEST_RUN` in modalità *live frames*.

5. Un core per il nodo successivo, una coda per il nodo

Il contatore d'uscita gira su un core suo, così che sul core del nodo resti solo il lavoro del nodo. E il nodo ha **una sola coda d'ingresso**, servita da un solo thread: con una coda per ogni thread del generatore il nodo avrebbe più thread sullo stesso core, e quale coda viene servita lo deciderebbe lo scheduler — a pieno carico ognuno terrebbe il core per millisecondi mentre le altre code traboccano.

12.4 I due generatori

Il banco ha due modi di produrre traffico, e servono tutti e due.

pktgen: il traffico come lo produce Linux

pktgen è un modulo del kernel che fabbrica pacchetti come li fabbrica lo stack di rete: ognuno è una *skb*, la struttura con cui Linux descrive un pacchetto, con 64 byte liberi davanti ai dati. XDP però ne vuole 256, quindi il cavo virtuale, prima di consegnare il pacchetto al nodo, lo **ricopia** in una pagina nuova. È un lavoro che il nodo paga, e che su una scheda di rete con XDP nativo non esisterebbe.

Tre thread su tre core offrono circa 5,7 milioni di pacchetti al secondo; la coda del nodo ne accetta circa 4,6 (quanto regge *rxonly*, il programma che riceve e butta).

xdp_gen: pacchetti già grezzi

Il secondo generatore (`ipa/test/xdp_gen.py`) consegna al cavo virtuale pacchetti già nel formato di XDP, con `BPF_PROG_TEST_RUN` in modalità *live frames*: niente *skb*, niente copia. Un solo thread offre 12–15 milioni di pacchetti al secondo, ed è così che si usa: un thread per ogni coda del nodo (sezione 12.9).

Esempio: lo stesso programma con i due generatori

P2 costa **527 ns** di CPU per pacchetto con *pktgen* e **292 ns** con *xdp_gen* (stessa sessione, 1 ottobre). Il programma è lo stesso byte per byte: i 235 ns di differenza sono la copia della *skb* e il percorso dello stack, cioè il costo del *generatore*, pagato sul core del nodo. *Sopra la baseline* invece i due generatori danno quasi lo stesso costo della rete neurale: P2 233 contro 228 ns, P1.5 56 contro 59, P1 40 contro 52. Il trasporto cambia, il costo del programma no.

12.5 Com'era il test nella macchina virtuale, e com'è adesso

Fino al 26 settembre questi banchi giravano in una macchina virtuale VirtualBox a quattro core virtuali. Dal 27 girano su Ubuntu installato direttamente sul portatile, in dual boot. Il programma misurato è lo stesso; è cambiato tutto quello che gli sta intorno.

Il problema della macchina virtuale: il cassiere che sparisce

Un core virtuale non è un core: è un thread del sistema che ospita la macchina virtuale, e quel sistema può fermarlo quando vuole, anche per qualche millisecondo, per far girare altro. Dentro la macchina virtuale non si vede e non si può impedire.

Per il banco è fatale, e il conto è semplice. La coda d'ingresso del nodo ha 256 posti. A 1,5 milioni di pacchetti al secondo un pacchetto arriva ogni $1/1,5 \cdot 10^6 \approx 0,67$ microsecondi, quindi la coda si riempie in

$$256 \times 0,67 \mu s \approx 170 \mu s.$$

Se il core del nodo sparisce per più di 170 microsecondi — e un core virtuale lo fa — la coda trabocca e i pacchetti vengono respinti, *per quanto veloce sia il programma*. Nell'immagine del supermercato: il cassiere è bravissimo, ma ogni tanto esce dalla cassa per qualche secondo, e intanto il nastro si riempie e i clienti vengono mandati via.

Nella macchina virtuale questo dava due strade, entrambe cattive:

- **Generatore e nodo su core separati:** il nodo perdeva fra l'8 e il 70% dei pacchetti *a qualunque ritmo*, anche lentissimo. La perdita misurava le assenze del core, non il programma.

- **Generatore e nodo sullo stesso core:** la coda si svuota subito e non si perde niente, ma il core fa il lavoro di tutti e due. Il ritmo misurato è $1/(t_{\text{generatore}} + t_{\text{programma}})$, e il collo di bottiglia è sempre il generatore.

Si usava la seconda, sottraendo la baseline per togliere il costo del generatore. Il risultato, su quattro esecuzioni identiche nell'arco di mezz'ora:

sopra la baseline	macchina virtuale (18 settembre)		dual boot (28 settembre)	
	costo	variazione	costo	variazione
P1 statica	~82 ns	160%	38 ns	0,5%
P1.5	~116 ns	104%	53 ns	0,6%
P2	~430 ns	19%	226 ns	0,5%
P3	~670 ns	15%	364 ns	0,3%

Macchina virtuale: generatore e nodo sullo stesso core, 512 byte, variazione = (massimo – minimo)/media su quattro esecuzioni. Dual boot: core separati, 64 byte, variazione fra i giri della stessa sessione.

Le cifre assolute delle due colonne **non si confrontano**: nella macchina virtuale la frequenza del processore era quella che capitava, e il metodo era diverso. Si confronta la **variazione**. Per P1 e P1.5 era più grande del valore stesso: una volta P1 costava 164 ns, un'altra 32. Il loro ordine si invertiva da un'esecuzione all'altra, e l'unica cosa onesta da scrivere era “stanno sotto i 200 ns e non si distinguono”. Oggi variano meno dell'1%, e la differenza fra le due — 15 ns — si misura.

Che cosa ha cambiato il dual boot

- **Core veri, ognuno col suo compito.** Generatore sui core 10, 1 e 3, nodo sul core 6, nodo successivo sull'8. Nessun altro processo, nessuna interruzione e nessun thread del kernel sul core del nodo (capitolo 13).
- **Il core del nodo non dorme mai.** Gli stati di sonno profondi sono spenti: il risveglio dal più profondo costa 310 microsecondi, più dei 170 che la coda concede. Nella macchina virtuale anche sotto capacità si respingeva l'1–2% dei pacchetti; oggi **zero**.
- **Frequenza fissa e misurata:** 3 494–3 495 MHz sul core del nodo durante ogni misura di traffico, letta dai contatori del processore.
- **Si possono separare i core senza perdere**, quindi il generatore non è più dentro la misura: il ritmo a cui il nodo satura è quello del nodo.

Esempio: lo stesso punto misurato tre volte

Nella macchina virtuale, il banco del bit rate ha misurato lo stesso punto — stessa pipeline, stesso ritmo — in tre giri diversi, e ha perso il 58%, lo 0,2% e il 7,7% dei pacchetti. Tre risposte diverse alla stessa domanda: nessuna era una misura del programma. Sul dual boot, cinque giri allo stesso punto danno lo stesso inoltro massimo entro lo 0,4–2%.

12.6 I limiti del test, con esempi

Il dual boot toglie il rumore. Non toglie i limiti di un banco che sta tutto su un portatile: vanno conosciuti, perché ognuno dice quali numeri si possono citare e quali no.

1. Il generatore può non bastare, o intralciare

Se il nodo è più veloce di quanto il generatore riesca a produrre, il banco misura il generatore: con un thread, pktgen si ferma a 2,8 milioni di pacchetti al secondo e xdp_gen a 12–15. Aggiungere thread sembra la soluzione, ma più thread che scrivono nella *stessa* coda si intralciano, e rallentano anche il nodo che la svuota: è la sezione 12.9.

Esempio: il “tetto del generatore” che non era del generatore

Con xdp_gen a tre thread rxonly arrivava a 6,89 milioni al secondo, la baseline a 7,11, P1 a 6,34, P1.5 a 6,23: tutti contro un soffitto di circa 7 milioni. Sembrava il limite del generatore. Invece, con un thread solo, la baseline elabora 12,4 milioni e rxonly 14,9: il soffitto era la coda condivisa da tre scrittori. P2 e P3, che saturano molto prima, non ne risentivano.

Con pktgen l'effetto è piccolo (circa 17 ns per pacchetto, uguali per tutte le pipeline) e tutte e cinque saturano: è per questo che la tabella principale del capitolo viene da pktgen a tre thread, dove anche rxonly satura.

2. Contare all'uscita sottostima il nodo

Il nodo successivo, che conta i pacchetti arrivati, ha un core solo, e anche lui ha un tetto: circa 4,4–4,9 milioni al secondo con xdp_gen. Se si misura il nodo contando *ciò che arriva dall'altra parte*, quando il nodo è più veloce dell'uscita si misura l'uscita.

Esempio: la baseline che sembrava lenta

Con xdp_gen la baseline elabora 7,1 milioni di pacchetti al secondo, ma all'altra parte ne arrivano 4,4. P1 ne elabora 6,3 e ne arrivano 4,8. Contando gli arrivi, la baseline sembra più lenta di P1 — impossibile, perché fa strettamente meno lavoro. Più pacchetti la baseline spinge verso l'uscita, più l'uscita, già satura, ne perde. Per questo il controllo di validità del banco (“la baseline dev'essere la più veloce”) conta i pacchetti *elaborati dal nodo*, la stessa misura con cui il banco calcola il costo, e non quelli arrivati all'uscita.

Con un solo scrittore per coda anche all'uscita (un core del nodo che inoltra, sezione 12.9) il nodo successivo tiene il passo: riceve 12 milioni di pacchetti al secondo senza perderne. Con due core sul nodo serve una coda d'uscita per ciascun core, e un core d'uscita per ciascuna coda: allora arrivano tutti, anche 29 milioni al secondo (sezione 12.10).

3. Il cavo è virtuale, e tutto sta su un processore

Il “ricevere” misurato è prendere il pacchetto dalla coda di un cavo virtuale, non dalla memoria di una scheda di rete; “inoltrare” è accodarlo sul cavo successivo, non scrivere un descrittore di trasmissione. Generatore, nodo e nodo successivo sono core dello stesso chip e condividono cache e memoria. Il portatile non ha una scheda di rete cablata che supporti XDP nativo.

Esempio: 217 ns per pacchetto di sola ricezione con pktgen e 145 con xdp_gen sono il prezzo di questo percorso. Su una scheda vera sarebbero altri numeri, e le cifre assolute di questo capitolo con loro. I **confronti** fra pipeline no: il percorso è lo stesso per tutte, e si sottrae.

4. Un solo tipo di pacchetto per volta

In una misura tutti i pacchetti sono uguali: stesso modello, stesso TTL, stessa classe decisa. Il processore impara in fretta un flusso così (il predittore dei salti, la cache), e un traffico misto costerebbe un po' di più. Le classi si misurano una alla volta, ed è così che si è visto che scartare costa più che inoltrare.

5. Le latenze si leggono solo al minimo

Il tempo di attraversamento si raccoglie in un istogramma a potenze di due: i percentili sono il bordo superiore della casella, con risoluzione di un fattore 2. Esempio: la mediana a basso carico è "1 024 ns" per baseline, P1 e P1.5 e "2 048 ns" per P2 e P3 — due caselle, non due misure. Il **minimo** invece è esatto (249, 282, 292, 460, 584 ns), ed è l'unico che si cita.

6. Il punto esatto in cui comincia la perdita è instabile

Il banco cerca anche il ritmo più alto a perdita nulla, restringendo per tentativi. Quella ricerca oscilla: fra un giro e l'altro dà valori diversi fino al 64%, perché al confine basta un'esitazione di pochi microsecondi per perdere un pacchetto. La saturazione invece varia fra lo 0,2 e il 2%. Si citano quindi la saturazione e il carico comune, non il confine; il bit rate lo descrive per gradini (sezione seguente).

7. Il portatile scalda

Oltre 3,5 GHz non regge una misura intera, e anche a 3,5 capita che il processore nel suo insieme tocchi per un attimo la soglia di temperatura, di solito mentre clang compila sugli altri core. Il banco lo registra: il 28 settembre è successo 3 volte durante la misura della latenza e mai durante quelle di throughput e di bit rate, e il core del nodo è rimasto a 3 495 MHz per tutto il tempo.

12.7 Il risultato: quanto regge il nodo

Con pktgen, pacchetti da 64 byte, tre giri per tre ripetizioni:

Pipeline	milioni di pacchetti/s	variazione fra i giri	sopra la baseline
solo ricezione (rxonly)	4,61	0,2%	—
baseline (nessuna rete)	3,31	0,9%	—
P1 statica	2,94	0,5%	+38 ns
P1.5 (hardcoded)	2,81	0,6%	+53 ns
P2 (template)	1,89	0,5%	+226 ns
P3 (modular)	1,50	0,3%	+364 ns

La **baseline** riceve, legge l'intestazione, decrementa il TTL e inoltra. Tutto ciò che le pipeline aggiungono sopra è il **costo della rete neurale**: circa 40–50 ns quando i pesi sono scritti nel programma (P1, P1.5), circa 220 ns quando la forma della rete è letta da una tabella (P2), circa 370 ns quando anche la profondità lo è (P3).

Come si legge la colonna di destra, con un esempio. La baseline regge 3,31 milioni di pacchetti al secondo, cioè spende $10^9 / 3,31 \cdot 10^6 = 302$ ns per pacchetto; P2 ne regge 1,89 milioni, cioè 529 ns. La differenza, con le cifre non arrotondate $528,6 - 302,5 = 226$ ns, è il tem-

po che P2 spende a eseguire la rete neurale: tutto il resto — ricevere, leggere l'intestazione, inoltrare — lo fanno tutte e due allo stesso modo.

E sotto la capacità? Offrendo a tutte lo stesso carico — 1,37 milioni di pacchetti al secondo, il 91% di quanto regge la più lenta — **nessuna pipeline respinge un solo pacchetto**, e fra ingresso e uscita se ne perdono al massimo lo 0,02%. Un nodo sotto la sua capacità consegna tutto.

P1 e P1.5 si separano

2,94 contro 2,81 milioni al secondo, con variazioni fra i giri dello 0,5 e dello 0,6%: congelare il nodo vale circa 15 nanosecondi sul percorso vero (340 contro 356 ns per pacchetto). Piccolo, ma fuori dal rumore. Lo stesso ordine lo danno il banco del bit rate (2,94 contro 2,84) e il cronometro del solo programma (68 contro 77 ns).

La ReLU senza bivio, sul traffico vero

Nel banco a pacchetto ripetuto P1 ha perso circa 5 ns con la ReLU senza bivio (capitolo 8). Qui no: sopra la baseline P1 costava 38 ns il 27 settembre e costa 38 ns oggi, P1.5 53 e 53. Il motivo è il predittore dei salti del processore: quando lo stesso pacchetto passa milioni di volte di fila impara da che parte va ogni bivio, e un bivio indovinato costa quasi niente — per questo toglierlo, nel banco, sembrava un costo. Sul percorso vero il resto del lavoro (ricevere, copiare, inoltrare) pesa molto di più, e la differenza sparisce nel rumore.

12.8 Il costo del nodo, con pacchetti grezzi

Con il generatore a pacchetti grezzi (`xdp_gen`, sezione 12.4), un thread solo, un core del nodo e il nodo successivo su un core suo — uno scrittore per coda, sezione 12.9 — il tempo di CPU che il nodo spende per ogni pacchetto, dalla presa del pacchetto alla consegna all'uscita, è:

Pipeline	ns di CPU per pacchetto	sopra la baseline	con pktgen
P1	117	+52	+38
P1.5	124	+59	+53
P2 (template)	292	+228	+226
P3 (modular)	432	+367	+364
baseline	65	—	—

1 ottobre, 3 giri a 64 byte (il 30 settembre entro l'1%). La colonna `pktgen` è la tabella del 28 settembre. La baseline, con un thread, non satura: il suo valore è indicativo. A 512 e 1 514 byte P2 e P3 costano uguale entro il 3%.

I costi sopra la baseline sono quasi gli stessi dei due generatori: la rete neurale costa lo stesso, comunque arrivi il pacchetto.

- **La taglia del pacchetto non conta:** a 64, 512 e 1 514 byte il costo è lo stesso entro il 3%. Il nodo guarda solo l'intestazione.
- **La porta d'uscita non conta:** le cinque classi che inoltrano costano uguale entro l'11%.
- **Scartare costa di più che inoltrare:** la classe DROP costa circa 50 ns in più delle altre (P1 173 contro 117, P1.5 177 contro 124, P3 480 contro 427). Quando il pacchetto viene scartato la sua pagina di memoria si restituisce sul core del nodo; quando viene inoltrato, su quello del nodo successivo. È un effetto del banco, non della pipeline.

12.9 Uno scrittore per coda

La domanda

Con `xdp_gen` a tre thread, `rxonly`, la baseline, P1 e P1.5 si fermavano tutte intorno a 7 milioni di pacchetti al secondo, mentre il generatore ne offriva 34–39 milioni e l'80% veniva respinto. Il limite era del generatore o del nodo? La prova naturale è cambiare il numero di thread: se il limite è del generatore, più thread lo alzano.

Con meno thread il nodo elabora di più

Thread del generatore	<code>rxonly</code>	baseline	P1.5
1	14,9	12,4	6,9
2	9,8	7,3	6,3
3	7,4	7,2	6,2

Milioni di pacchetti elaborati al secondo, un core del nodo, `xdp_gen`. Misura presa a batteria: le differenze arrivano al doppio, e le misure con l'alimentatore qui sotto dicono la stessa cosa.

Il limite non era né il generatore né il programma: era **la coda**. Tre thread scrivono nella stessa coda da 256 posti, e per farlo devono passarsi un lucchetto e le stesse righe di cache; il core del nodo, che dalla stessa coda legge, ne subisce il rallentamento. È come una cassa dove tre fornitori scaricano merce nello stesso punto: il cassiere non lavora peggio, ma gli spazi sono contesi.

La prova con due code

Con due core sul nodo ci sono due code. Con due thread, uno per coda, la baseline elabora 13,0 milioni al secondo; con tre thread, due dei quali finiscono sulla stessa coda, 7,6. Lo stesso nodo, cambia solo quanti scrivono in ogni coda. (Le cifre sono del 30 settembre, con una sola coda d'uscita per i due core: in assoluto sono basse anche per la contesa in uscita, sezione 12.10, ma la pagano tutte e due le righe.) P1 e P1.5 guadagnano il 45%, P2 il 6%, P3 l'1%: il danno è grande per i programmi leggeri e quasi nullo per quelli il cui limite è davvero il programma.

Con `pktgen`, che riempie la coda più lentamente, la contesa costa circa 17 ns per pacchetto, uguali per tutte le pipeline: i costi sopra la baseline non cambiano. Lo stesso vale **all'uscita**, quando il nodo ha più core: è la sezione 12.10.

Un solo scrittore per coda, in ingresso e in uscita: più thread di generatore non aiutano, peggiorano. Con una scheda di rete vera la regola è rispettata da sé: la coda di ricezione la riempie la scheda, un solo scrittore, e i pacchetti degli altri host arrivano in fila sul cavo. Resta quello che conta, la coda piena davanti a un nodo troppo lento.

Il tempo di CPU, non il tempo che passa

Con un solo thread `xdp_gen` spedisce a raffiche: la raffica riempie la coda e il resto viene respinto, poi fra una raffica e l'altra il nodo svuota la coda e *aspetta*. Il core del nodo lavora così l'87% del tempo, anche mentre respinge il 70–80% dei pacchetti. Dividere il tempo per i pacchetti elaborati conterebbe anche l'attesa: P2 risulterebbe 337 ns invece di 292.

Per questo il banco misura quanto il core del nodo è occupato, nella stessa finestra in cui conta i pacchetti, e dà il costo come **tempo di CPU per pacchetto**: occupazione per il tempo diviso i pacchetti. Con pktgen il nodo è occupato al 100% e le due cose coincidono: le cifre della tabella principale sono tempi di CPU veri.

Scartare o inoltrare

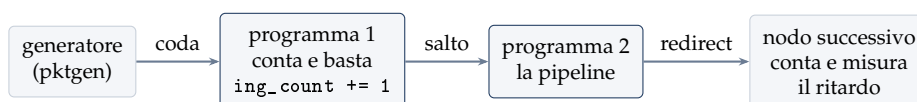
Un gemello di rxonly che invece di scartare inoltra, nella stessa sessione a tre thread, elabora il 4% in più (7,53 contro 7,21 milioni al secondo); la baseline, 7,21, *non* supera rxonly. Il “baseline più veloce di rxonly” del 28 settembre era una differenza di pochi punti dentro la coda contesa. La misura pulita è quella per classe: scartare costa al nodo circa 50 ns in più che inoltrare.

12.10 Al crescere del bit rate

La domanda: al crescere del traffico offerto la coda si riempie, il tempo di attraversamento cresce, e poi si perde. Se si perde *all’ingresso* del programma, il collo di bottiglia è il programma; se si perde *dopo*, è la trasmissione. Per saperlo bisogna contare quanti pacchetti il programma riceve e quanti ne rilancia.

Due programmi sullo stesso aggancio

Un’interfaccia porta un solo programma XDP. Due programmi sullo stesso aggancio si ottengono in **catena**, con lo stesso salto fra programmi che le pipeline usano già:



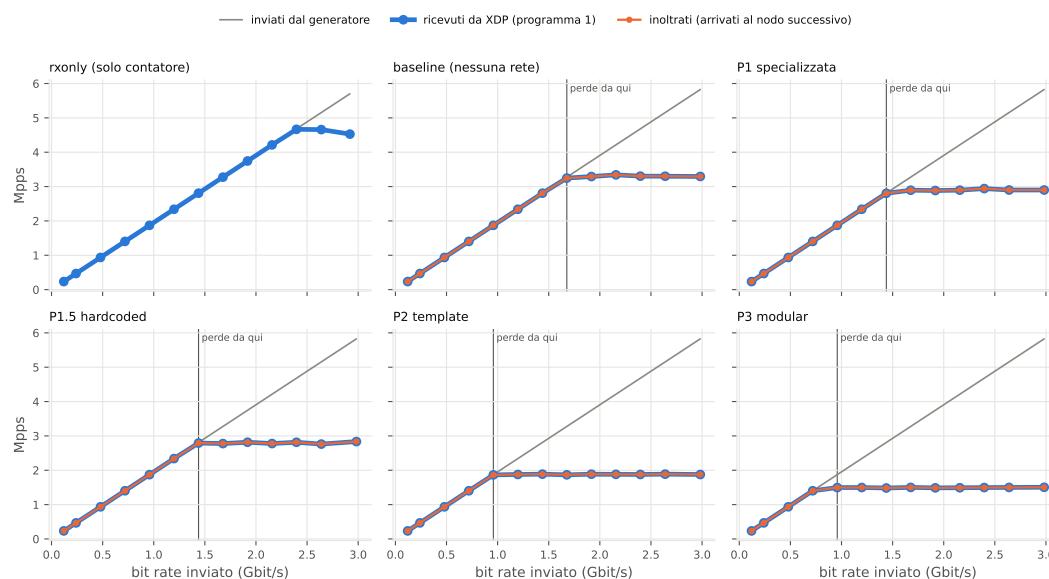
Il primo programma è attaccato all’interfaccia, conta ogni pacchetto e salta al secondo, che è il dispatcher della pipeline, identico a quello di produzione.

Perché in catena. Il kernel tiene, per ogni interfaccia, *un* programma XDP nell’aggancio nativo: è come è fatto XDP, non un limite di questo progetto. I modi di averne più di uno sono tutti catene. Quello standard è libxdp, che aggancia un programma *dispatcher* con alcuni posti liberi e ci inserisce gli altri; quello usato qui è il salto fra programmi (*tail call*) che le pipeline già usano, senza dipendenze in più. (Per il punto di aggancio di tc il kernel dal 6.6 hanno anche una lista nativa di programmi; per XDP no.) Il costo del salto in più si misura, più sotto: pochi nanosecondi, dentro il rumore. Così si hanno tre conteggi per ogni valore di traffico: **inviati** (dal generatore), **ricevuti** (dal programma 1) e **inoltrati** (arrivati al nodo successivo). Inviati meno ricevuti è la perdita *prima* del programma; ricevuti meno inoltrati quella *dopo*. Un terzo metodo, rxonly, è il contatore da solo: dice quanto regge la sola ricezione allo stesso ritmo, e separa il collo del programma da quello del banco.

Il banco misura tredici valori di traffico, da 0,12 a 3 Gbit/s a pacchetti da 64 byte, calibrati sul tetto di ricezione della macchina, cinque giri ciascuno. Il ritardo è quello di tutto il percorso: dal timbro che il generatore scrive nel pacchetto all’arrivo al nodo successivo.

Ricevuti e inoltrati

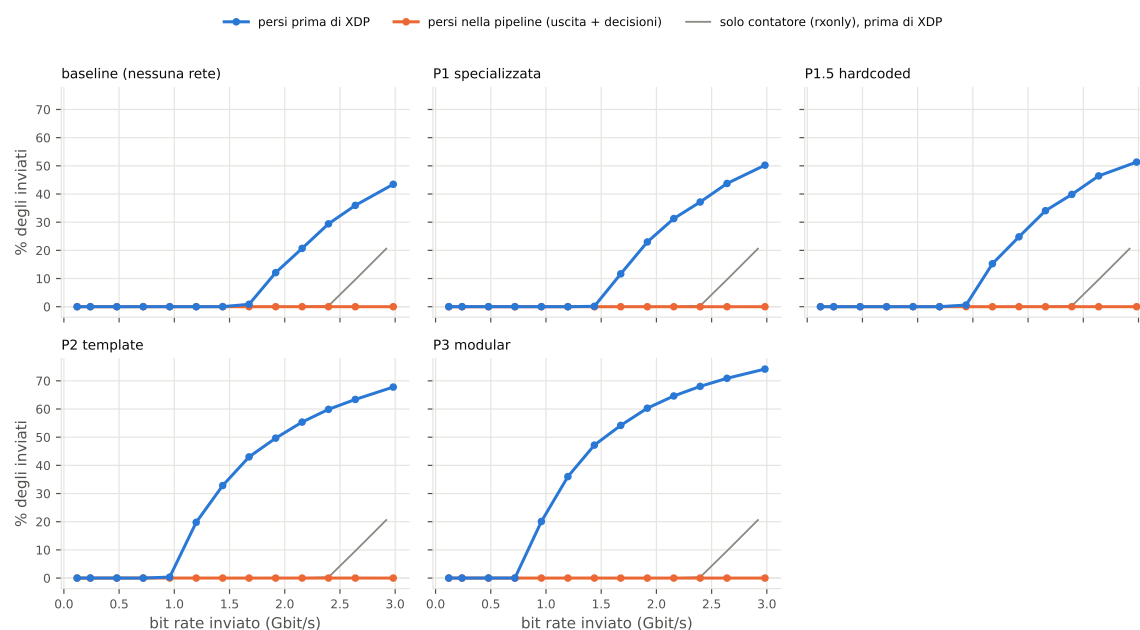
Pacchetti al secondo contro bit rate inviato — frame 64 B



Inviati sopra i ricevuti: si perde prima di XDP (coda d'ingresso piena, collo il programma o la ricezione). Ricevuti sopra gli inoltrati: si perde dopo XDP (uscita, collo trasmissivo) o il modello scarta. Barre: min-max fra i giri.

Ogni pannello ha tre curve. Finché il nodo regge sono sovrapposte: tutto ciò che si invia viene ricevuto e inoltrato. Oltre il suo limite la linea grigia (inviati) continua a salire e le altre due si fermano **insieme**, alla capacità della pipeline: 3,3 milioni al secondo per la baseline, 2,9 per P1, 1,9 per P2, 1,5 per P3. Che ricevuti e inoltrati restino sovrapposte è la risposta alla domanda: **il programma rilancia tutto ciò che riceve**, e ciò che manca non è mai entrato.

Perdita contro bit rate inviato — frame 64 B



La perdita prima di XDP oltre quella del solo contatore e' del programma; quella dopo XDP e' trasmissiva. Mediana fra i giri.

La stessa cosa come perdita: la curva blu (persi prima di XDP) sale, la rossa (persi dentro la pipeline o all'uscita) resta a zero, al massimo lo 0,02% degli inviati. La linea grigia è il

solo contatore: comincia a perdere solo oltre 2,4 Gbit/s, molto dopo le pipeline, quindi la loro perdita non è della ricezione ma del programma.

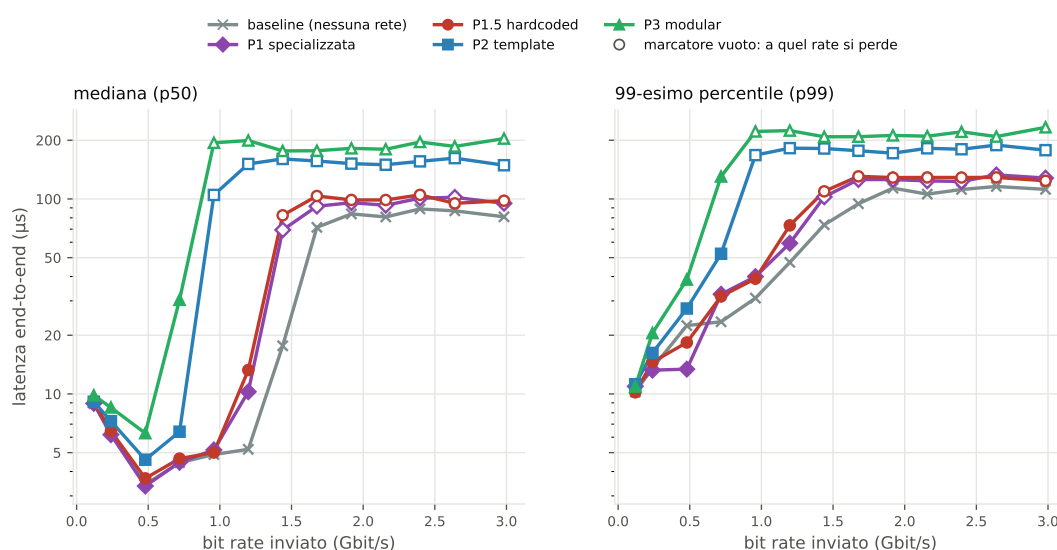
Pipeline	pulita fino a	perde da	inoltre massimo
baseline	1,44 Gbit/s	1,68 Gbit/s	3,34 M/s
P1 statica	1,20	1,44	2,94
P1.5	1,20	1,44	2,84
P2	0,72	0,96	1,89
P3	0,72	0,96	1,50

pacchetti da 64 byte; bit rate misurato sul collegamento; 28 settembre

Le prime due colonne sono **gradini**, non soglie esatte. I tredici valori sono spaziati di circa 0,24 Gbit/s, e “pulita fino a 0,72, perde da 0,96” vuol dire che la perdita comincia da qualche parte fra i due. Esempio: P2 inoltra al massimo 1,89 milioni di pacchetti al secondo, cioè $1,89 \cdot 10^6 \times 64 \times 8 \approx 0,97$ Gbit/s; il gradino da 0,96 cade proprio sul suo limite, e perde. P3 (1,50 milioni, $\approx 0,77$ Gbit/s) perde allo stesso gradino, pur essendo più lenta: fra 0,72 e 0,96 non c'è un valore intermedio che le separi. A distinguerle è la terza colonna.

Il ritardo

Latenza end-to-end contro bit rate inviato — frame 64 B



Timbro di pktgen -> nodo successivo, corretto per l'attesa di pktgen; mediana fra i giri. Scala logaritmica.

Il ritardo fa quello che la domanda prevede: resta di pochi microsecondi finché il nodo regge, poi **salta di un ordine di grandezza** esattamente dove la pipeline comincia a perdere (marcatori vuoti), e da lì resta piatto. Il salto è la coda d'ingresso che si riempie: 256 posti davanti a un programma che serve un pacchetto ogni 300–670 ns fanno fra 80 e 170 microsecondi di attesa, vicino al livello a cui le curve si fermano — circa 85 μs per la baseline, 100 per P1, 150 per P2, 190 per P3; il resto è il percorso fino al nodo successivo. Oltre, la coda non può crescere: è piena, e ciò che arriva in più viene respinto. Il ritardo si ferma e cresce la perdita.

Un dettaglio a basso carico

Al valore più basso (0,12 Gbit/s) la mediana è di circa 9 μ s, più che a 0,5 Gbit/s (3–5 μ s): la curva scende prima di salire. Non è la correzione per l’attesa del generatore, che a quel ritmo vale 12 μ s ed è quasi uguale alla distanza fra due pacchetti dello stesso thread. Il minimo resta intorno a 1 μ s a tutti i ritmi. La spiegazione più probabile, **non verificata**: con poco traffico i core del percorso che il banco non tiene svegli — il nodo successivo — vanno a dormire, e una parte dei pacchetti aspetta il loro risveglio. Non tocca la conclusione, che sta nel salto a destra.

Perché il collo di bottiglia è il programma

È quello che ci si deve aspettare, per come funziona XDP. Il programma non ha una coda sua: gira dall’inizio alla fine su ogni pacchetto, sul core che riceve, e finché non ha finito quel core non prende il pacchetto successivo. Se i pacchetti arrivano più in fretta di quanto il core li finisca, si accumulano nella coda *davanti* al programma, e quando è piena vengono respinti. La perdita all’ingresso è quindi il segno di un nodo più lento del traffico, e il nodo è il suo programma.

Resta da escludere che il limite sia la sola ricezione, che gira sullo stesso core. Lo dice rxonly, il contatore da solo: regge fino a circa 2,4 Gbit/s, e a 3 Gbit/s perde il 21%. Allo stesso ritmo la baseline perde il 44%, P2 il 68%, P3 il 74%: la perdita *in più* rispetto al solo contatore (23, 47 e 54 punti) è del programma, e cresce con il suo costo. L’uscita invece non limita mai: dopo il programma si perde lo 0,002% al massimo.

Due precisazioni. Per la baseline il “programma” è quasi solo lettura dell’intestazione e redirect: il suo limite è il costo fisso di un nodo XDP, non una rete neurale. E il nodo ha **una sola coda e un solo core** per scelta, per misurare il costo per core: una scheda di rete vera distribuisce i pacchetti su più code e più core (RSS), e la capacità del nodo cresce con il numero di core, mentre il costo per pacchetto resta quello misurato qui.

Perché la capacità cresca davvero con i core, i core non devono contendersi niente di scritto a ogni pacchetto. Per questo i contatori delle pipeline (pacchetti inoltrati, persi, scartati, e uno per classe) sono **per-core**: ogni core incrementa la sua copia, senza istruzioni atomiche, e chi legge somma le copie. Un contatore unico incrementato da più core costringerebbe i core a passarsi a ogni pacchetto la stessa riga di cache. Su un core solo il guadagno è piccolo: circa 8 ns per pacchetto per i programmi leggeri (la baseline passa da 3,31 a 3,40 Mpps, P1 da 2,94 a 3,01), niente di misurabile per P2 e P3. Le altre cifre del capitolo restano quelle misurate con i contatori di prima.

Lo si verifica dando al nodo due core invece di uno, ciascuno con la sua coda d’ingresso e la sua coda d’uscita, e con pktgen a quattro thread, due per coda d’ingresso:

Programma	1 core (Mpps)	2 core (Mpps)	rapporto	ns per core
rxonly	4,69	9,29	1,98	215
baseline	3,41	6,72	1,97	298
P1	3,00	5,96	1,99	335
P1.5	2,86	5,60	1,96	357
P2	1,90	3,68	1,94	543
P3	1,46	2,81	1,93	711

Stessa sessione, 1 ottobre, alimentatore, macchina non disturbata. 1 core: tre thread; 2 core: quattro, due per coda.

Con due core ogni programma elabora il doppio (fra 1,93 e 1,99 volte): il secondo core non rallenta il primo. Per core, il solo contatore, la baseline, P1 e P1.5 costano quanto su

un core entro 8 ns; P2 e P3 il 3–4% in più (543 contro 527, 711 contro 684 ns), da cui i loro rapporti un po' più bassi. Con i pacchetti grezzi di `xdp_gen` il risultato è lo stesso: due core elaborano il 97–99% del doppio, e il tempo di CPU per pacchetto di ogni core (121, 126, 297, 439 ns per P1, P1.5, P2, P3) è quello misurato su un core solo (117, 124, 292, 432) entro il 3,5%. Per un nodo con N code vale quindi N volte la capacità misurata su un core, finché generatore e scheda reggono.

Il raddoppio si vede solo se ogni coda ha un solo scrittore anche **all'uscita**. Fino al 30 settembre il banco dava a ogni porta d'uscita una coda sola: il cavo virtuale sceglie la coda dal numero del core che inoltra, e con una coda i due core del nodo scrivevano nello stesso anello, svuotato da un solo core del nodo successivo.

Esempio: la baseline a due core con una coda d'uscita

Con `xdp_gen` la baseline elaborava 13,15 milioni di pacchetti al secondo e al nodo successivo ne arrivavano 4,2: il 49% si perdeva dopo il programma, e ogni core spendeva 152 ns per pacchetto invece di 65. Con tre code d'uscita (il core 6 scrive nella 0, il core 8 nella 2) gli elaborati salgono a 19,15, ma un core d'uscita solo ne riceve circa 10 milioni e il resto si perde ancora. Con un core d'uscita per coda la baseline elabora 28,8 milioni al secondo, arrivano tutti, e il costo per core torna a 68 ns (65 su un core, indicativo). Con `pktgen`, che arriva a rate più bassi, la coda d'uscita condivisa costava 12–20 ns per pacchetto per core: i rapporti erano 1,91–1,98 invece di circa 2.

Anche il generatore va diviso in parti uguali. Con tre thread `pktgen` su due code una coda riceve da due thread e l'altra da uno solo, circa 2,8 milioni al secondo: abbastanza per saturare i core dei programmi, non quello di `rxonly`, che ne riceve 4,7. Il solo contatore si fermava così a 7,55 milioni (1,61 volte), con un core fermo il 40% del tempo; con quattro thread, due per coda, arriva a 9,29 (1,98 volte).

Per tutte e cinque la perdita è **all'ingresso**: il collo di bottiglia è il programma eBPF, non la trasmissione. Il ritardo cresce fino a una perdita, come previsto, e si ferma al livello della coda piena.

Il primo programma della catena costa poco, e si misura invece di supportarlo: il kernel tiene il tempo di ogni programma attaccato, e con e senza il contatore davanti la differenza va da -2 a $+5$ ns, entro la variazione fra le ripetizioni della misura stessa.

12.11 Che cosa questo numero non dice

I limiti del banco sono nella sezione 12.6. Uno in più riguarda il modello:

- **Un modello solo.** Il costo sul traffico vero è misurato sul modello depositato (65-4-4-7); come cresce con modelli più grandi lo dicono i capitoli precedenti, a livello di programma.

La macchina di laboratorio, sotto controllo

Tutte le misure di questo quaderno sono fatte sulla macchina di laboratorio, senza strati di mezzo: un portatile Lenovo IdeaPad Slim 5 con un **Intel Core Ultra 7 155H**, Ubuntu 24.04 installato direttamente sul disco (dual boot), kernel 6.8. Questo capitolo spiega che cosa serve perché le misure si ripetano.

13.1 Un processore che non sta fermo

Il 155H è **ibrido**: tre tipi di core diversi sullo stesso chip.

tipo	CPU	frequenza	note
P-core (prestazioni)	0–11	base 1,4, turbo 4,5–4,8 GHz	6 core fisici, due thread ciascuno fuori dalla cache condivisa
E-core (efficienza)	12–19	base 0,9, turbo 3,8 GHz	
LP E-core	20–21	base 0,7, turbo 2,5 GHz	

Lasciato a sé stesso, un banco su questa macchina finisce dove capita: una regola generica per scegliere i processori (un terzo al nodo, il resto al generatore) darebbe quattordici thread di generatore e metterebbe il nodo sui core di efficienza, e il desktop girerebbe sul gemello del core misurato. La frequenza va da 0,4 a 4,8 GHz secondo il carico, e i core si addormentano in uno stato da cui il risveglio costa 310 microsecondi: più di quanto la coda d'ingresso da 256 posti copra a un milione di pacchetti al secondo.

E il portatile **scalda**. Un solo P-core salito a 4,5–4,8 GHz porta il processore da 51 a 99 °C in due secondi, e il chip riduce la frequenza per difendersi (*throttling*).

13.2 Le condizioni, messe e tolte

`ipa/test/host_conditions.py` mette la macchina in condizioni note per la durata di un banco, e le riporta com'erano alla fine.

- **Ruoli**: nodo, nodo successivo e generatore su P-core fisici *diversi*, un thread per core; il gemello di ciascuno resta a riposo; il core della CPU 0 resta al sistema. Qui: nodo sulla CPU 6, nodo successivo sull'8, generatore su 10, 1 e 3.
- **Frequenza fissa** sui core del banco (governor *performance*, minimo uguale al massimo), e **misurata** dai contatori del processore (APERF/MPERF).
- **Tetto sui core del sistema** (2,5 GHz): il turbo del desktop scalderebbe il chip fino a rallentare anche i core del banco.
- **Niente sonno profondo** sui core del banco: spenti gli stati con risveglio oltre un microsecondo.
- **Isolamento**: desktop, servizi, interruzioni e thread del kernel spostabili confinati sulle CPU del sistema.

- **Frequenza della cache condivisa fissa**, profilo energetico *performance*, watchdog spento.

Ogni valore si salva *prima* di cambiarlo, in un file in memoria. A fine run — anche con Ctrl-C o un errore — si riscrive tutto all'indietro; un run ucciso lascia il file, e il run successivo ripristina per primo. Un test sul kernel vero (`test_host_kernel.py`, 25 controlli) applica le condizioni, le rilegge dal kernel, verifica che un processo del desktop giri solo sulle CPU del sistema, misura la frequenza sotto carico, ripristina e confronta la macchina valore per valore con com'era — anche dopo aver ucciso il processo a condizioni applicate.

Durante ogni misura un **monitor** legge gli interventi di throttling, la temperatura, la frequenza vera del nodo e le interruzioni del firmware. Una finestra misurata fuori dalle condizioni dichiarate viene marcata e contata, non scartata in silenzio.

13.3 La frequenza scritta non è la frequenza vera

La frequenza *base* del processore, 1,4 GHz, è l'unica garantita. Il sistema la accetta e la rilegge: 1 400 minimo, 1 400 massimo. I contatori del processore dicono un'altra cosa:

Con 1 400 MHz scritti nel sistema, il core gira a **circa 2 000 MHz** — fisso e stabile, ma non a quella frequenza. Da 3 000 MHz in su scritto e vero coincidono entro 10 MHz.

Se il monitor si fidasse del numero scritto, a 1,4 GHz marcherebbe come disturbate 38 finestre su 39. La frequenza di riferimento è quindi quella **misurata** all'avvio, su ogni core del banco; il numero scritto resta accanto, per confronto.

13.4 Quale frequenza

Più alta la frequenza, più alto il throughput — finché il portatile regge. La risposta si è misurata:

frequenza vera	rxonly	baseline	P2	macchina
2,0 GHz	2,65	2,00	1,14	pulita
3,0 GHz	3,93	2,89	1,66	pulita
3,5 GHz	4,37	3,22	1,86	pulita
4,0 GHz	4,84	3,55	2,09	pulita in 30 s; in 5 minuti fino a 98 °C
4,5 GHz	5,17	3,54	2,04	throttling su 28 finestre su 39

milioni di pacchetti al secondo inoltrati, 64 byte

A 4,5 GHz il throughput smette di crescere: il chip si difende. A 4 GHz un run corto è pulito, uno da cinque minuti porta il processore a 98 °C e il core del nodo stesso in throttling. **3,5 GHz** è la frequenza più alta che la macchina regge per un run intero, ed è la frequenza di default del banco.

La frequenza pesa quasi in proporzione, ma non del tutto: fra 2 e 4 GHz i cicli per pacchetto crescono dell'8–13%, perché la cache condivisa e la memoria non accelerano con il core. Quello che non si muove è il **rapporto fra pipeline**: P2 regge il 57–59% della baseline a ogni frequenza. Le cifre assolute si citano con la loro frequenza; i rapporti no.

13.5 Il risultato: misure che si ripetono

banco	variazione
programma da solo (test_suite, fra sette prove)	2–6%
throughput a saturazione (fra tre giri)	0,2–0,9%
inoltre massimo al crescere del bit rate (fra cinque giri)	1,5–2,5%

E i pacchetti respinti con il nodo sotto capacità sono **zero** (nella macchina virtuale erano l'1–2% anche per il programma che non fa niente): la causa era il risveglio del core, e il core del nodo non dorme.

13.6 Prima e dopo: che cosa è cambiato

Fino al 26 settembre i banchi giravano in una macchina virtuale VirtualBox a quattro core virtuali, sullo stesso portatile. Dentro non si vedeva la frequenza, non si sceglieva il core fisico, e il sistema sotto poteva fermare un core virtuale per millisecondi. Il 27 settembre tutto è passato su Ubuntu installato direttamente sul portatile (dual boot), con le condizioni di questo capitolo. Confrontando le misure di prima con quelle di oggi ("oggi" è il 28 settembre, con la ReLU senza bivio del capitolo 8; il dettaglio sul test con il generatore è nel capitolo 12):

Che cosa è migliorato

	prima	oggi
programma da solo, latenza (baseline / P1 / P2 / P3)	28 / 76 / 311 / 445 ns	18 / 52 / 195 / 315 ns
dispersione fra le prove	13–30%	2–6%
costo della rete su traffico vero, sopra la baseline	P2 ~430, P3 ~670 ns; P1 e P1.5 non misurabili	38 / 53 / 226 / 364 ns
variazione fra i run	15–19% per P2 e P3, oltre il 100% per P1 e P1.5	0,2–0,9% fra i giri
respinti con il nodo sotto capacità	1–2% anche per il solo contatore	zero
bit rate: lo stesso punto in giri diversi	58% / 0,2% / 7,7% di perdita	inoltre massimo entro lo 0,4–2%
P1 contro P1.5	indistinguibili, l'ordine si inverte fra i run	2,94 contro 2,81 M/s, sempre nello stesso ordine
P3, costo per strato in più	~70 ns	~53 ns
frequenza del core	né visibile né controllabile	fissa e misurata: 3 494 MHz

Il guadagno vero non è la velocità, è la **ripetibilità**: la dispersione scende di un ordine di grandezza. Differenze che prima sparivano nel rumore — P1 contro P1.5, una porta in più o in meno, scartare contro inoltrare — ora si misurano. Le latenze assolute di prima e di oggi invece non si confrontano: prima la frequenza era quella che capitava.

Che cosa è peggiorato, o è diventato un limite

- **P2 oltre due strati nascosti non si caricava più.** Nella macchina virtuale si caricava fino a sei strati (da 14 668 a 18 801 istruzioni); qui il verificatore la rifiutava da tre strati in

su. La causa non era il kernel ma il codice: ogni ReLU era un salto condizionale, e già a uno strato il verificatore percorreva 800 000 istruzioni su un limite di un milione. Con la ReLU senza salto (28 settembre) P2 carica fino a venti strati.

- **P1 a ~1 200 pesi, lo stesso.** Sull'ingresso di IPA prima si caricavano tutte le forme, dalla larga alla profonda (122–293 ns); qui passava solo quella a otto strati. Stessa causa di P2, stessa correzione: con la ReLU senza salto il tier B carica in tutte le forme.
- **Il portatile scalda.** Oltre 3,5 GHz non regge un run intero, quindi il banco non può usare il turbo.
- **Serve più preparazione:** la frequenza scritta nel sistema non è quella vera, clang non trova un header senza un percorso in più, e sul sistema nuovo PyTorch non è installato, quindi la parte userspace di `test_suite.py` non parte finché non lo si installa.

Nessuna misura è peggiorata. Sono peggiorati due **limiti di caricamento**, e sembravano dipendere dal kernel. Le statistiche del verificatore (`diag_verifier.py`) hanno detto altro: i programmi lavoravano già vicino al limite di complessità, per colpa di un salto condizionale in ogni ReLU, e una piccola differenza fra kernel bastava a superarlo. Tolto il salto, il margine torna ampio (P2 a sei strati usa il 14% del limite).

13.7 Come si rifà tutto

Due script rifanno ogni misura di questo quaderno, nelle stesse condizioni. I CSV vanno in `results/`, i registri in `/tmp/ipa_logs`, e i grafici del quaderno si rigenerano dai CSV in `docs/figures/`:

```
bash ipa/test/remeasure_all.sh      # programma, correttezza, analisi parametrica
bash ipa/test/remeasure_traffic.sh  # traffico vero, semantica, soffitti, topologie
```

Altri modelli, altre reti

14.1 La domanda

Fino a questo capitolo quasi ogni numero viene da un modello solo, il 65-4-4-7 addestrato su Germany50. L'unica eccezione era il confronto a tre vie del capitolo sui modelli sintetici, che però prova solo la correttezza: la suite del kernel, la consegna sul fabric e i banchi di traffico caricavano sempre e solo il checkpoint.

E non per scelta: in molti punti la forma del checkpoint era scritta nel codice. Qualche esempio:

- P3 veniva caricata con gli strati scritti a mano, [(65,4), (4,4), (4,7)];
- il fabric provava 6 bit di stato dei link e si dichiarava nodo 7;
- il significato delle classi veniva sempre da `model_meta.json`, quello del checkpoint;
- con `sudo` i pesi venivano sempre da `weights.json`, qualunque file si passasse.

Un modello diverso poteva quindi girare con i pesi, o con le classi, di un altro, senza che nessun test se ne accorgesse. L'obiettivo di questo capitolo: **lo stesso test, per qualsiasi modello e qualsiasi rete**, e un errore chiaro quando la combinazione non ha senso.

14.2 Scenario e modello

Le due parole hanno un significato preciso.

- **Lo scenario è la rete:** quattro numeri in `topologies/<nome>/topology_config.json`, cioè quante interfacce ha al massimo un nodo, quanti nodi ha la rete, quante code ha ogni interfaccia e con che TTL i pacchetti ci entrano.
- **Il modello è la rete neurale:** pesi, forma, ingressi (ognuno con la sua larghezza e la sua scala) e significato di ogni classe.

Scenario	interf.	nodi	code	TTL	modelli che ci girano
germany50	6	52	4	30	checkpoint, ipa_like, deep, sparse, sparse_hetero_11
germany50_ttl16	6	52	4	16	ipa_ttl16
synth_small	3	8	–	16	small
synth_ones	3	4	–	8	ones
synth_large	8	100	–	64	large
synth_mixed	4	12	1	30	mixed

Oltre a Germany50 ci sono le reti dei modelli sintetici. Non sono topologie reali: sono le dimensioni per cui ciascun modello sintetico è stato generato.

14.3 Quando un modello può girare su una rete

Ogni ingresso del modello prende la sua larghezza da una dimensione della rete: lo stato dei link e la porta d'ingresso dalle interfacce, l'identità del nodo dai nodi, l'occupazione

delle code dalle code. E il TTL il modello lo legge diviso per il TTL iniziale con cui è stato addestrato (sezione 2.3 nel capitolo sulla matematica). Quindi:

Un modello gira su una rete solo se **ogni suo ingresso è largo quanto la dimensione della rete da cui dipende** e se **la scala del suo TTL è il TTL iniziale della rete**.

Esempio: il checkpoint ha il one-hot del nodo largo 52. Su una rete da 30 nodi metà delle sue colonne non corrisponderebbe a nessun nodo, e il test si ferma subito con

```
odi: il modello ne vuole 52, la rete ne ha 30
```

Esempio con il TTL: `ipa_ttl16` ha la stessa forma del checkpoint, ma è addestrato con il TTL diviso per 16. Su `Germany50`, dove i pacchetti partono da 30, vedrebbe valori fino a quasi 2 invece che fino a 1, fuori da tutto ciò su cui è stato addestrato. Per questo gira sulla sua rete, `germany50_ttl16`.

C'è poi un secondo filtro, fra modello e *pipeline*. Ogni pipeline ha dei tetti fissati quando il programma viene compilato:

- **P2**: al massimo 8 neuroni per strato, strati dal terzo in poi larghi come il secondo, 1 024 pesi in tutto;
- **P3**: al massimo 8 neuroni per strato, 16 strati, 2 048 pesi;
- **P1**: nessuno, perché il programma si genera per quel modello.

Il modello large (117-8-8-9) ha 1 097 pesi e un primo strato da 117 per 8: P2 e P3 non lo possono caricare. Non è un errore del test: la combinazione è **non applicabile**, ed è contata a parte.

14.4 Come è costruito

Quattro pezzi, ognuno con un compito solo.

1. `model_source.py` **carica qualsiasi modello nello stesso formato**: il checkpoint, un altro .pt, un preset sintetico (`synth:deep`) o una cartella. Il significato delle classi è quello che il modello dichiara. Contiene anche il controllo modello contro rete.
2. `pipeline_limits.py` **dice se una pipeline regge una forma**, con i numeri presi dai moduli che compilano i programmi, così i due non possono divergere.
3. `pipeline_setup.py` **carica un modello in una pipeline**: baseline, P1, P1.5, P2, P3. Sono i costruttori che usava già il confronto a tre vie, spostati in un posto comune.
4. `model_under_test.py` **legge** `-model` e `-topology` e li passa ai processi figli con due variabili d'ambiente.

Il punto importante è che cosa succede *senza* `-model`: niente. Ogni funzione controlla per prima cosa se c'è un modello sotto test, e se non c'è esegue le stesse righe di prima. Tutti i numeri dei capitoli precedenti restano validi, e si possono rifare con gli stessi comandi.

```
sudo python3 ipa/test/test_suite.py --only kernel --model synth:deep
sudo python3 ipa/test/test_fabric.py --model synth:small
sudo python3 ipa/test/bench_throughput.py --mode compare --model synth:deep
sudo python3 ipa/test/bench_bitrate.py --model synth:deep
```

Senza `-topology` viene scelta la rete compatibile (`Germany50` se lo è).

14.5 Che cosa controllano i test, adesso

Passare un modello diverso ha costretto a rendere i controlli più precisi in tre punti.

L'azione, non solo la classe. Prima la verifica per TTL accettava come corretto soltanto un redirect: col checkpoint, fra TTL 2 e 6, la classe è sempre un inoltro. Un altro modello può decidere DROP proprio lì. Ora il test controlla che il programma abbia scelto la classe del riferimento *e* ne abbia eseguito l'azione: redirect per un inoltro, XDP_DROP per uno scarto, XDP_PASS per una classe inutilizzata.

Tutte le classi che il modello sa decidere. Il fabric cercava un caso per ogni classe cambiando stato dei link e TTL, con nodo e porta d'ingresso fissi. Col checkpoint basta, perché è lo stato dei link a spostare la sua decisione. Coi pesi casuali dei sintetici conta anche il resto, e la ricerca trovava da 1 a 3 classi. Adesso, per le classi mancanti, la ricerca cambia anche porta d'ingresso, nodo e code, e ogni caso scrive i suoi valori nelle mappe prima di spedire il pacchetto.

Modello	prima	adesso
ipa_like, ipa_ttl16	3, 4, 5	2, 3, 4, 5
small	0, 1, 2	0, 1, 2, 3
mixed	0, 1, 3	0, 1, 2, 3
large	4, 8	1, 4, 6, 8
deep / sparse / ones	0, 1, 4 / 1, 5 / 0	uguale

Le classi che mancano ancora il modello non le decide mai: su 200 000 ingressi casuali deep sceglie solo 0, 1 e 4, e ones, che ha tutti i pesi uguali a 1, sceglie sempre la classe 0. È una proprietà del modello, non un limite del test.

Il traffico misura un inoltro. I generatori mandano pacchetti con TTL 32. Col checkpoint, con tutti i link attivi, quel pacchetto viene inoltrato; con large invece viene scartato, e il banco misurerebbe lo scarto chiamandolo inoltro. Il banco allora cerca lo stato dei link che porta a un inoltro, lo scrive nella mappa e lo stampa:

```
traffico di large con link_state=00111000, TTL 32 -> classe 4 (FORWARD)
```

14.6 La prova che la strada nuova non cambia niente

Il checkpoint si può passare anche esplicitamente, `-model checkpoint`: va per la strada nuova. Confrontato con la strada di sempre:

- pesi, scala e significato delle classi identici; il riferimento Python dà la stessa classe e lo stesso logit su 3 000 ingressi;
- il sorgente di P1 e la foglia di P2 sono identici *byte per byte*;
- il riferimento generico, che la strada nuova usa per tutti, coincide con quello scritto per il 65-4-4-7 su 20 000 ingressi;
- sul traffico vero: 3,37 / 2,98 / 2,81 / 1,87 / 1,47 Mpps contro 3,40 / 3,01 / 2,87 / 1,91 / 1,49 del giorno prima. Anche la baseline, che nessun modello tocca, perde l'1%: è variazione fra sessioni.

E la correttezza sui modelli nuovi:

- nel kernel, ogni scenario per ogni modello compatibile per ogni pipeline, 40 ingressi a testa, sotto due identificativi di modello: **110 su 110**, più large non applicabile a P2 e P3;
- suite kernel e fabric su nove modelli: tutto PASS.

14.7 I risultati

Sul traffico vero, un core, stesso banco del capitolo 12. Milioni di pacchetti al secondo e, fra parentesi, i nanosecondi per pacchetto in più rispetto alla baseline:

Modello	forma	baseline	P1	P1.5	P2	P3
checkpoint	65-4-4-7	3,37	2,98 (+39)	2,81 (+58)	1,87 (+237)	1,47 (+385)
deep	65-4-4-4-7	3,31	2,87 (+46)	2,76 (+60)	1,78 (+261)	1,35 (+438)
small	15-4-4	3,30	3,01 (+29)	2,88 (+45)	2,11 (+172)	1,74 (+271)
mixed	18-6-5	3,28	3,02 (+27)	2,90 (+40)	2,06 (+181)	1,67 (+293)
large	117-8-8-9	3,30	2,61 (+80)	2,54 (+90)	—	—

Tre cose si leggono subito.

- **Uno strato in più costa soprattutto a P3:** deep ha uno strato più del checkpoint e costa +24 ns su P2, +53 ns su P3, +7 ns su P1. P3 paga ogni strato con un salto fra programmi e una lettura dei suoi pesi; P1 ha tutto dentro lo stesso programma.
- **Ingressi più piccoli abbassano P2 e P3** di 60–110 ns: il primo strato, che è il più largo, scorre meno colonne.
- **Un ingresso più grande pesa su P1:** large costa a P1 il doppio del checkpoint, 80 ns contro 39.

Nel kernel, con `BPF_PROG_TEST_RUN`, si vede anche una cosa che il traffico non può separare: **P2 e P3 costano per la forma, P1 per i valori dei pesi**. Checkpoint, `ipa_like`, `ipa_ttl16` e `sparsed` hanno la stessa forma: P2 e P3 costano uguale per tutti (194–198 e 312–322 ns), mentre P1.5 va da 38 ns (`sparsed`, pesi a zero tolti in compilazione) a 51. Queste latenze però sono state prese col portatile a batteria: sono indicative finché non si rimisurano con l'alimentatore.

Il bit rate su deep ripete il comportamento del checkpoint: per tutte le pipeline la perdita comincia *all'ingresso*, quindi il collo di bottiglia resta il programma eBPF. Cambia solo dove: P3 è pulita fino a 0,48 Gbit/s invece che 0,72, perché lo strato in più la porta sotto 1,41 Mpps.

Il confronto fra le pipeline non è una proprietà del checkpoint: su cinque modelli l'ordine $P1 < P1.5 < P2 < P3$ resta lo stesso, e i costi si spostano come ci si aspetta dalla forma. La profondità pesa soprattutto su P3, l'ampiezza dell'ingresso su P1.

14.8 Che cosa non fa ancora

- **Tre marcature e latenza minima** (`-mode rates`, `-latency`) girano solo col checkpoint: usano versioni del programma con i timbri dentro, generate dal suo sorgente.
- **Una misura per modello** sul traffico vero, e le latenze del kernel dei sintetici prese a batteria.
- **Nessun modello sintetico è addestrato:** servono a provare le pipeline su forme diverse, non a dire qualcosa sull'instradamento.

I numeri di oggi, in una pagina

	baseline	P1 statica	P1.5	P2	P3
<i>Il nodo su traffico vero (capitolo 12), 64 byte</i>					
milioni di pacchetti al secondo	3,31	2,94	2,81	1,89	1,50
costo della rete (sopra la baseline)	—	38 ns	53 ns	226 ns	364 ns
pulita fino a (bit rate)	1,44	1,20	1,20	0,72	0,72 Gbit/s
<i>Il solo programma</i>					
$T_2 - T_1$ sul percorso vero (mini-mo)	34 ns	68 ns	77 ns	228 ns	340 ns
BPF_PROG_TEST_RUN, pesi del modello	14 ns	48 ns	53 ns	200 ns	314 ns
istruzioni del programma	135	606	1 019	15 089	12 288
<i>Cambiare modello su un nodo in servizio</i>					
caricare sul nodo	—	0,7–2,1 ms (AOT)		10–12 ms	
anche generare e compilare	—	66–120 ms		non serve	
serve un compilatore sul nodo?	—	no (compilato prima)		no	

Traffico vero e $T_2 - T_1$ misurati il 28 settembre, il programma da solo (BPF_PROG_TEST_RUN e istruzioni) il 29 con i contatori per-core; tutto con la ReLU senza bivio (capitolo 8), core del banco a 3,5 GHz. Il bit rate si misura su una griglia di valori calibrata sulla sessione: “pulita fino a” è l’ultimo punto della griglia senza perdite, non la capacità esatta.

In tre righe:

1. **Scrivere i pesi nel programma costa 40–50 ns per pacchetto; leggerli da una tabella 220–370 ns.** È il prezzo della configurabilità.
2. **Con l’oggetto precompilato, cambiare modello costa pochi millisecondi per tutte le pipeline.** Il vantaggio residuo di P2 e P3 è non dover compilare niente, da nessuna parte.
3. **Congelare anche il nodo (P1 statica) vale poco ma si vede:** circa 15 ns sul percorso vero, 5–9 ns sul solo programma, e la dimensione non dipende più dalla taglia della rete.

Quel che resta aperto

- **Il costo del nodo di baseline e rxonly** con pacchetti grezzi: con un thread per coda il generatore non le satura, e più thread intralciano la coda (sezione 12.9). Serve un generatore più veloce su un solo scrittore, o una scheda vera.
- **Tre marcature e latenza minima su modelli diversi:** oggi solo col checkpoint (capitolo 14).
- **Le latenze del kernel dei modelli sintetici** con l'alimentatore: quelle del capitolo 14 sono state prese a batteria.
- **Hardware vero:** una scheda di rete con XDP nativo e una seconda macchina, per un numero che non sia di un cavo virtuale.
- **Il tetto sull'uscita di P2:** lo strato finale è srotolato a 32 classi per un modello che ne ha 7. Abbassarlo ridurrebbe il programma ma *restringe l'insieme dei modelli rappresentabili*.
- **I salti fra programmi di P3:** tre per pacchetto, ed è lì (con le letture che li accompagnano) che sta la latenza.
- **La profondità massima di P2** fra 20 e 37 strati: il punto esatto in cui clang si ferma non è misurato.
- **Isolamento dal boot:** i core del banco sono isolati a sistema acceso; l'isolamento completo (tick e callback del kernel) richiede parametri di avvio.

I comandi

Tutti i comandi si lanciano dalla radice del repository. `sudo` serve ovunque si carichi un programma nel kernel o si tocchi un'interfaccia; senza `sudo` girano solo le prove che non escono da Python.

Prerequisiti

Linux con BCC installato; per l'oggetto AOT di P1 anche

```
sudo apt install clang libbbpf-dev libelf-dev zlib1g-dev libzstd-dev liblzma-dev
```

Il generatore di traffico (`pktgen`) il banco lo carica da sé.

17.1 La macchina

```
# topologia, piano dei ruoli e condizioni attuali (niente root)
python3 ipa/test/host_conditions.py --show

# un comando qualunque sul core del nodo, a condizioni applicate
sudo python3 ipa/test/host_conditions.py --run -- CMD ...

# se un run e' stato ucciso: rimette la macchina com'era
sudo python3 ipa/test/host_conditions.py --restore

# le condizioni verificate: su un /sys finto, e sul kernel vero
python3 ipa/test/test_host_conditions.py
sudo python3 ipa/test/test_host_kernel.py --bench

# tutte le misure del quaderno
bash ipa/test/remeasure_all.sh
bash ipa/test/remeasure_traffic.sh
```

17.2 Correttezza: il modello fa quello che deve

```
python3 ipa/test/test_suite.py
python3 ipa/test/test_class_semantics.py
python3 ipa/test/test_synth.py
python3 ipa/test/verify_synth_kernel.py --all --dry-run
```

17.3 Kernel: si carica, e quanto costa

```
sudo python3 ipa/test/host_conditions.py --run -- \
    python3 ipa/test/test_suite.py --only kernel
sudo python3 ipa/test/verify_multi_model.py
sudo python3 ipa/test/verify_per_model_semantics.py
sudo python3 ipa/test/verify_synth_kernel.py --all --n 300
```

```
sudo python3 ipa/test/verify_synth_kernel.py --all --n 300 --pipeline p2
sudo python3 ipa/test/verify_synth_kernel.py --all --n 300 --pipeline p3
```

17.4 Altri modelli, altre reti

```
python3 ipa/test/test_model_source.py
sudo python3 ipa/test/test_model_source.py --kernel
sudo python3 ipa/test/host_conditions.py --run -- \
    python3 ipa/test/test_suite.py --only kernel --model synth:deep
sudo python3 ipa/test/test_fabric.py --model synth:small -q
sudo python3 ipa/test/bench_throughput.py --mode compare --rounds 3 \
    --gen-cpus 10,1,3 --dut-cpus 6 --model synth:deep \
    --out results/models/throughput_deep
sudo python3 ipa/test/bench_bitrate.py --model synth:deep --rounds 3 \
    --out results/models/bitrate_deep
```

17.5 Datapath reale: il pacchetto attraversa davvero

```
sudo python3 ipa/test/test_fabric.py
sudo python3 ipa/test/test_fabric.py --method aot
sudo python3 ipa/test/test_fabric.py --sweep
sudo python3 ipa/test/netns_fabric.py --cleanup
```

17.6 Analisi parametrica: come cresce il costo

```
sudo python3 ipa/test/host_conditions.py --run -- \
    python3 ipa/test/bench_scaling.py --axis all --out results/
sudo python3 ipa/test/host_conditions.py --run -- \
    python3 ipa/test/bench_scaling.py --axis campaign --out results/
python3 ipa/test/bench_scaling.py --plot results/
sudo python3 ipa/test/bench_scaling.py --verify
sudo python3 ipa/test/bench_scaling.py --verify-ports
sudo python3 ipa/test/host_conditions.py --run -- \
    python3 ipa/test/bench_depth_vs_width.py
```

17.7 Il test reale

```
# throughput a saturazione e a carico comune (pktgen)
sudo python3 ipa/test/bench_throughput.py --mode compare --rounds 3 \
    --out results/throughput_3500

# il bit rate crescente: latenza, ricevuti, inoltrati, dove si perde
sudo python3 ipa/test/bench_bitrate.py --out results/bitrate_3500
python3 ipa/test/bench_bitrate.py --report results/bitrate_3500

# le tre marcature T1/T2/T3 su una scala di rate
sudo python3 ipa/test/bench_throughput.py --mode rates --frames 64 \
    --rounds 3 --rates 0.5,1,1.5,2,2.5,3

# pacchetti grezzi, nodo successivo su un core suo; per classe
sudo python3 ipa/test/bench_throughput.py --mode compare --generator xdp \
```

```

--egress-cpu auto --rounds 5
sudo python3 ipa/test/bench_throughput.py --mode compare --generator xdp \
--egress-cpu auto --per-class --rounds 3

# la finestra di misura e le formule, verificate senza kernel
python3 ipa/test/test_steady_window.py
python3 ipa/test/test_bitrate_math.py

# se un run si interrompe e lascia il generatore configurato
sudo python3 ipa/test/bench_throughput.py --cleanup

```

Come si legge l'uscita

Condizioni del test — in testa, e in env.csv: ruoli, frequenza scritta e misurata, temperatura, interventi di throttling durante il run.

Controllo di validità — la baseline deve essere la più veloce entro il 10% e rxonly non deve essere più lento della baseline. Se una delle due cose non vale, il run misura il banco e non le pipeline.

Finestre marcate — «macchina» (fuori dalle condizioni dichiarate), «raffica» o «gen» (il generatore non ha offerto il carico chiesto): contate e riportate, non scartate.

17.8 Costi specifici e oggetto precompilato

```

sudo python3 ipa/test/bench_tailcall_overhead.py
python3 ipa/methods/method4_hardcoded_aot.py --build-only # build, come utente
sudo python3 ipa/methods/method4_hardcoded_aot.py # verifica e bench
sudo python3 ipa/test/diag_verifier.py # statistiche del verificatore

```

17.9 L'ordine consigliato

Partendo da zero su una macchina nuova:

1. `python3 ipa/test/host_conditions.py -show` — com'è fatta la macchina, e dove andrebbe il banco?
2. `python3 ipa/test/test_synth.py` — il C generato per P1 calcola il modello? (`test_suite.py`, che controlla estrazione dei pesi e quantizzazione, richiede PyTorch)
3. `sudo python3 ipa/test/test_host_kernel.py` — le condizioni si mettono e si tolgono davvero?
4. `sudo python3 ipa/test/test_fabric.py` — il pacchetto attraversa davvero?
5. `bash ipa/test/remeasure_all.sh` e `bash ipa/test/remeasure_traffic.sh` — tutti i numeri.